

MySQL复习

MySQL复习

第1章 MySQL概述与安装配置

- 知识点1 安装配置MySQL
- 知识点2 mysql的命令行客户端的使用
- 知识点3 数据库的基本概念
- 知识点4 关系型数据库和非关系型数据库的区别

第2章 MySQL存储引擎和数据类型

- 知识点1 MySQL的存储引擎
- 知识点2 MySQL的数据类型
- 知识点3 MySQL的数据属性

第3章 MySQL数据库基础

- 知识点1 数据库的增加和删除操作
- 知识点2 数据表的操作
- 知识点3 数据的添加操作
- 知识点4 数据的修改操作
- 知识点5 数据的删除操作
- 知识点6 数据的查询操作
- 知识点7 数据高级查询操作（上）
- 知识点7 数据高级查询操作（下）
- 知识点8 MySQL中的常用函数

第4章 MySQL安全

- 知识点1 创建用户
- 知识点2 掌握给用户授权
- 知识点3 撤回用户权限
- 知识点4 删除用户
- 知识点5 限制访问资源
- 知识点6 MySQL索引操作
 - 索引优劣势：
 - 索引使用场景：
 - 内部原理讲解
 - B-树
 - B+树
 - MYISAM索引引擎
 - INNODB索引引擎

第5章 视图

- 知识点1 视图的基本概念
- 知识点2 创建视图
- 知识点3 修改视图
- 知识点4 使用视图
- 知识点5 删除视图

第6章 存储过程

- 知识点1 了解存储过程的基础知识
- 知识点2 创建存储例程的方式
- 知识点3 执行存储过程
- 知识点4 存储过程的删除

第7章 触发器

- 知识点1 前触发器和后触发器
- 知识点2 创建触发器

知识点3 查看已有触发器

知识点4 删除触发器

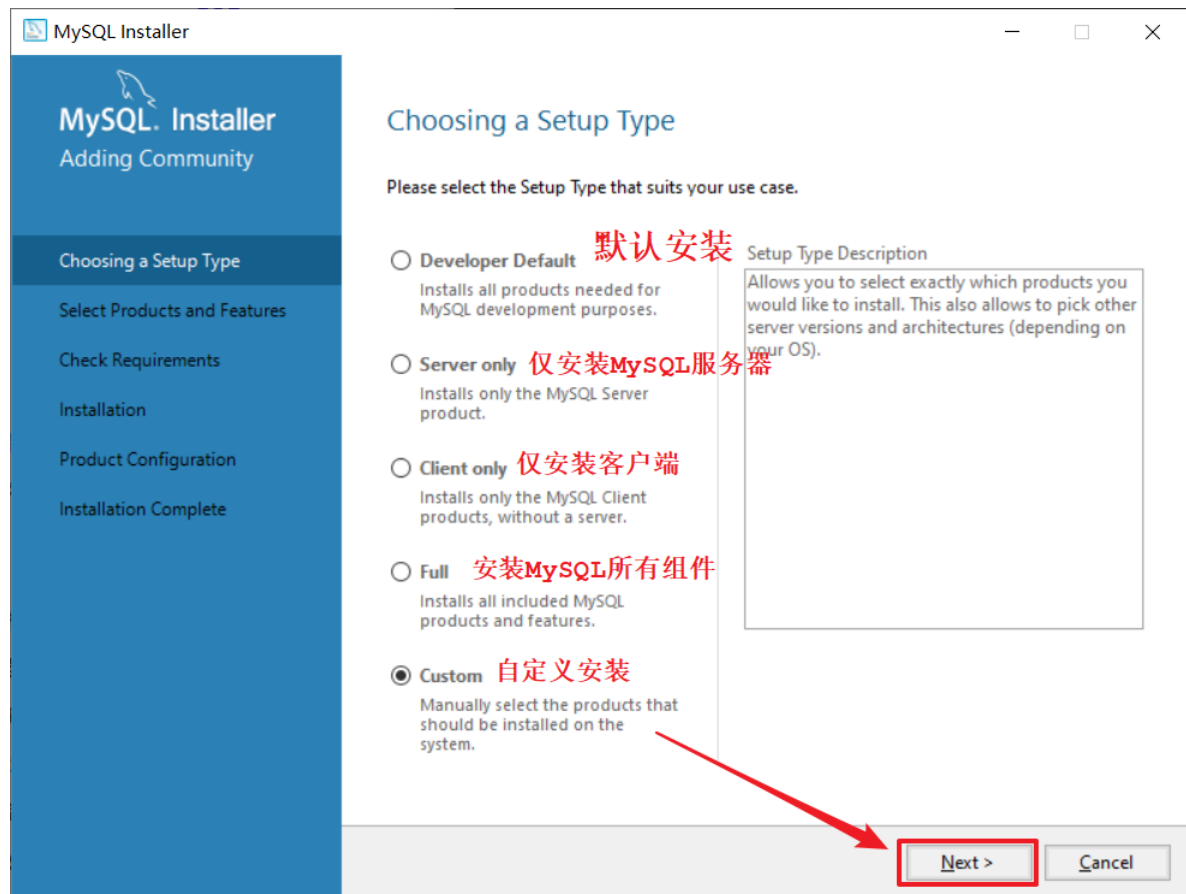
练习题

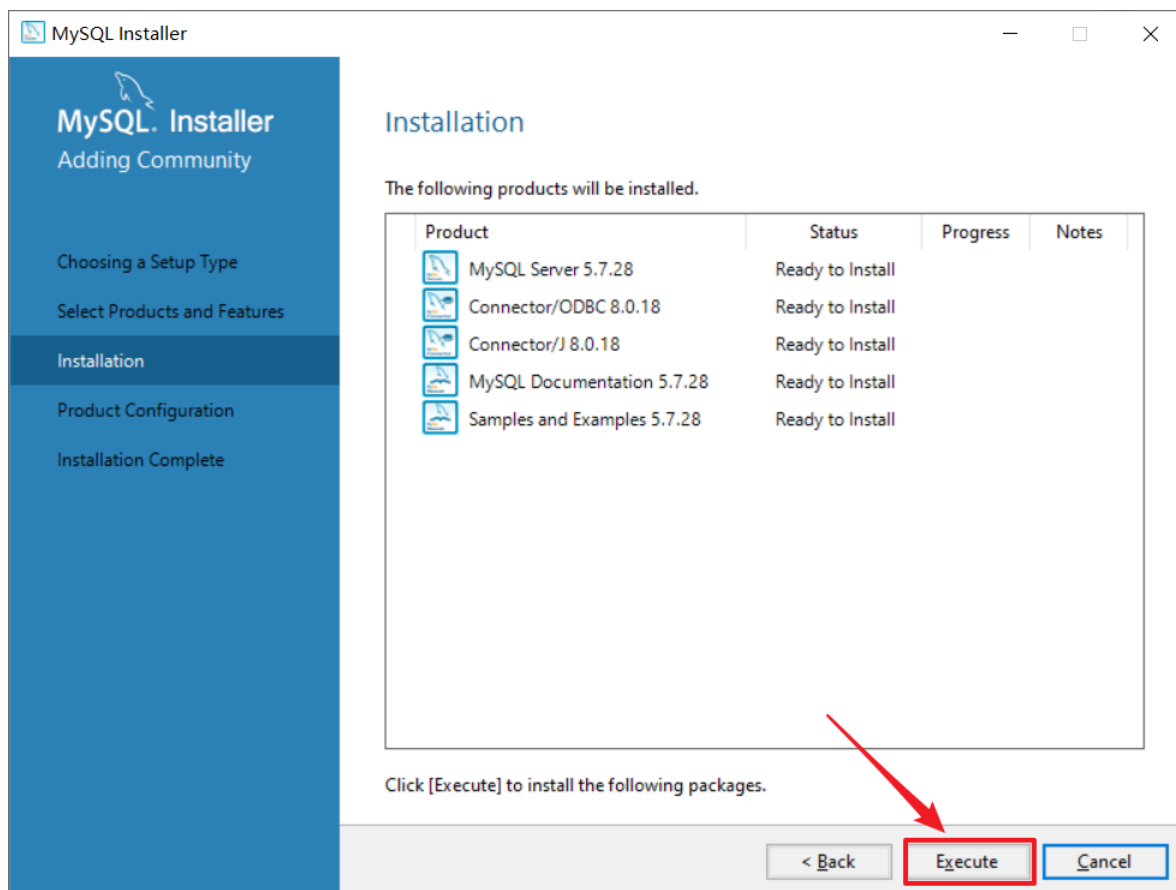
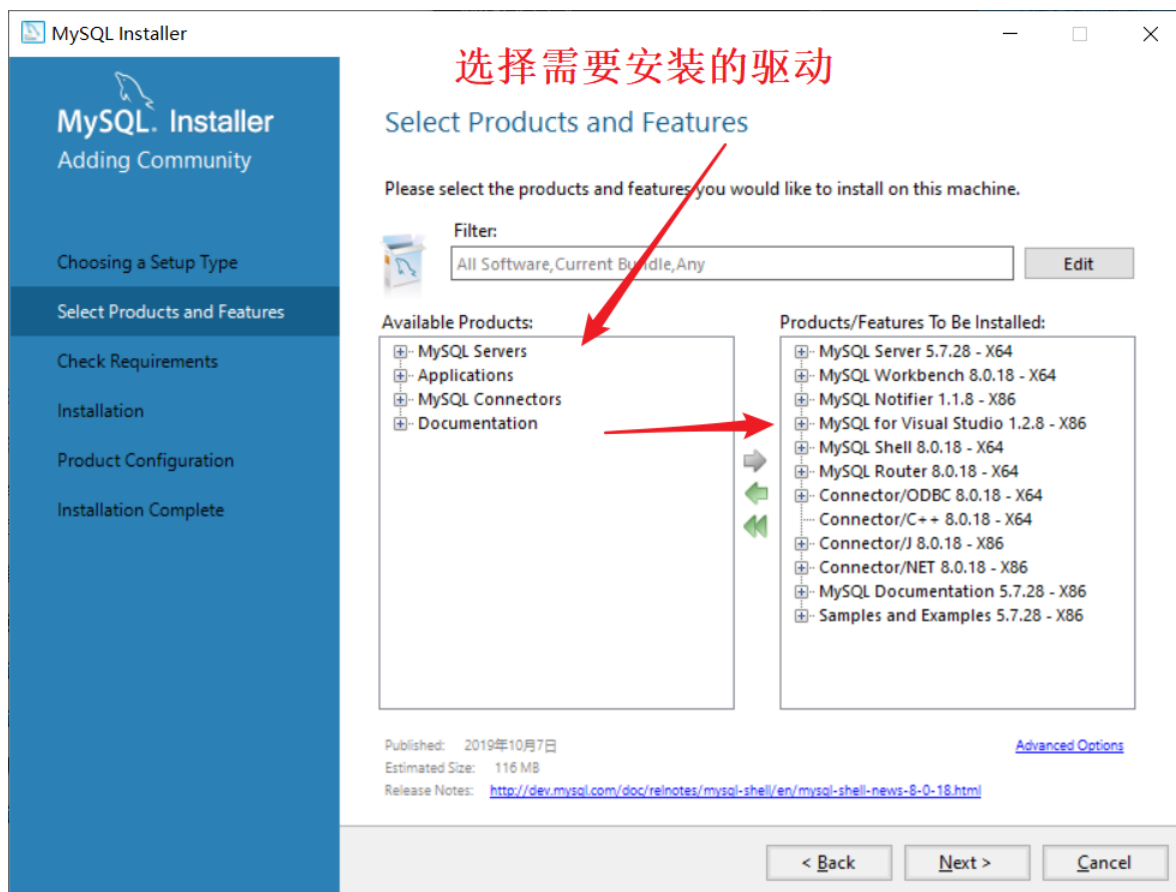
- 1、--- 给任职日期超过15年的员工加薪20%-- 注意:先备份表, 然后对备份表进行操作
- 2、--- 求部门名称中带“A”字符的部门员工的工资合计、 部门人数
- 3、--- 求出员工领导的薪水超过1700的员工名称和领导名称
- 4、--- 列出各种工作的最低工资及从事此工作的雇员姓名
- 5、--- 列出薪金高于在部门20工作的所有员工的薪金的员工 姓名和薪金、 部门名称
- 6、--- 列出至少有5个员工的所有部门
- 7、--- 求平均薪水的等级最低的部门的部门名称
- 8、--- 取得平均薪水最高的部门的部门编号

索引面试题

第1章 MYSQL概述与安装配置

知识点1 安装配置MySQL





MySQL. Installer
Adding Community

Choosing a Setup Type

Select Products and Features

Installation

Product Configuration

Installation Complete

Installation

The following products will be installed.

Product	Status	Progress	Notes
 MySQL Server 5.7.28	Complete		
 Connector/ODBC 8.0.18	Complete		
 Connector/J 8.0.18	Complete		
 MySQL Documentation 5.7.28	Complete		
 Samples and Examples 5.7.28	Complete		

Show Details >

< Back

Next >

Cancel

MySQL. Installer
Adding Community

Choosing a Setup Type

Select Products and Features

Installation

Product Configuration

Installation Complete

Product Configuration

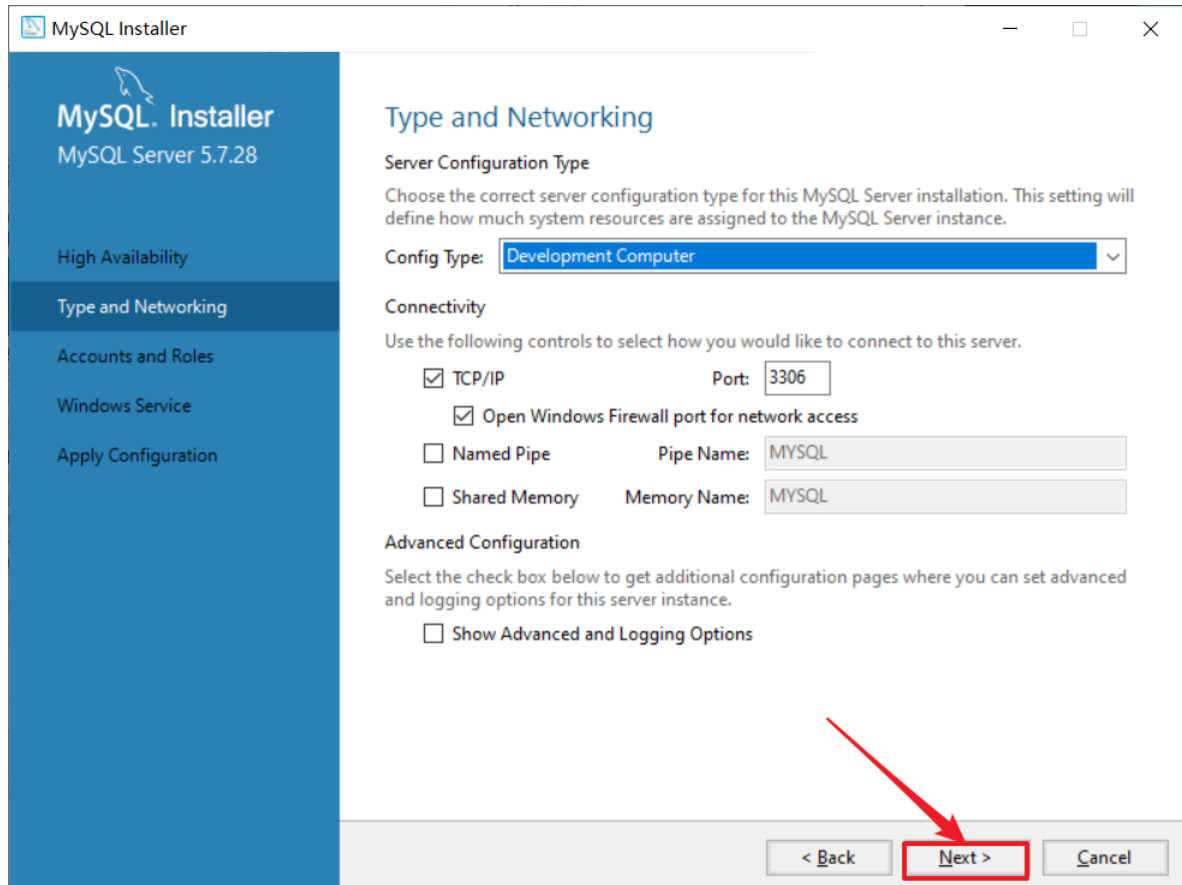
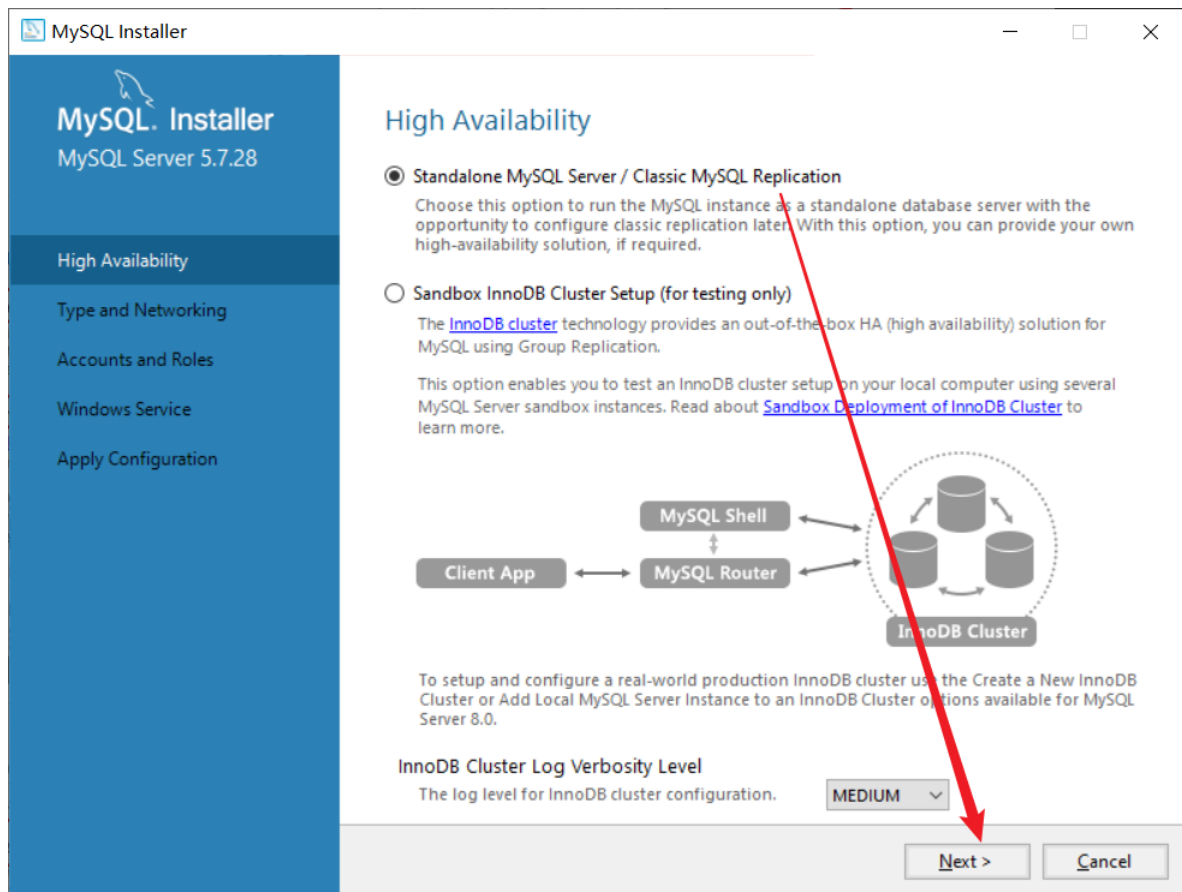
We'll now walk through a configuration wizard for each of the following products.

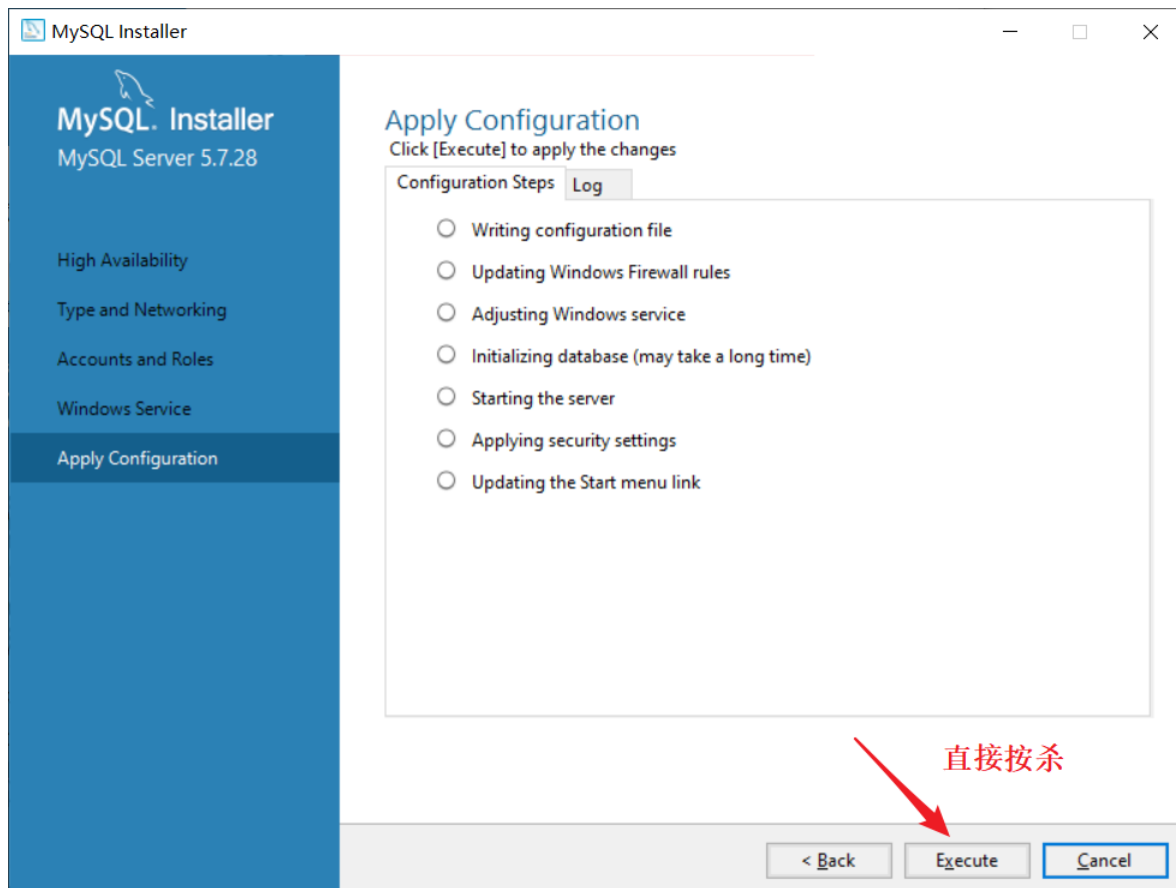
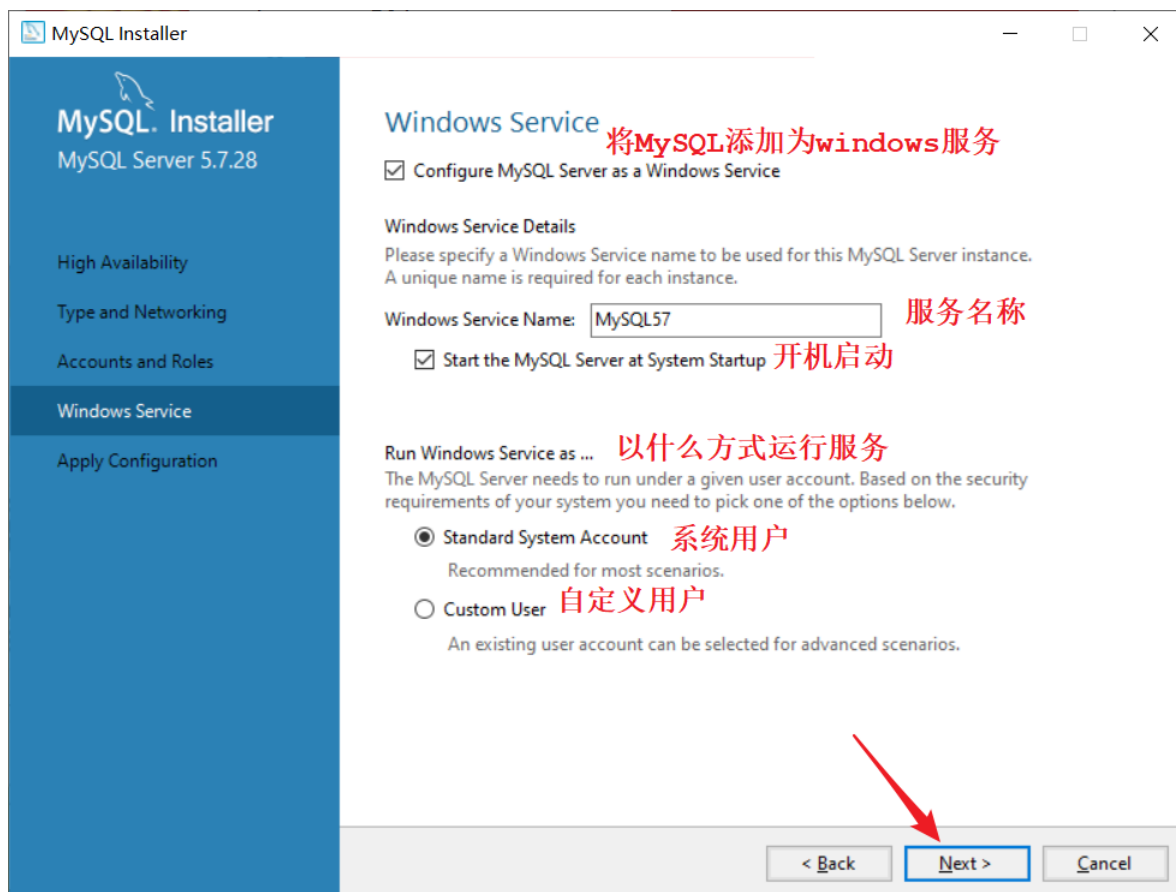
You can cancel at any point if you wish to leave this wizard without configuring all the products.

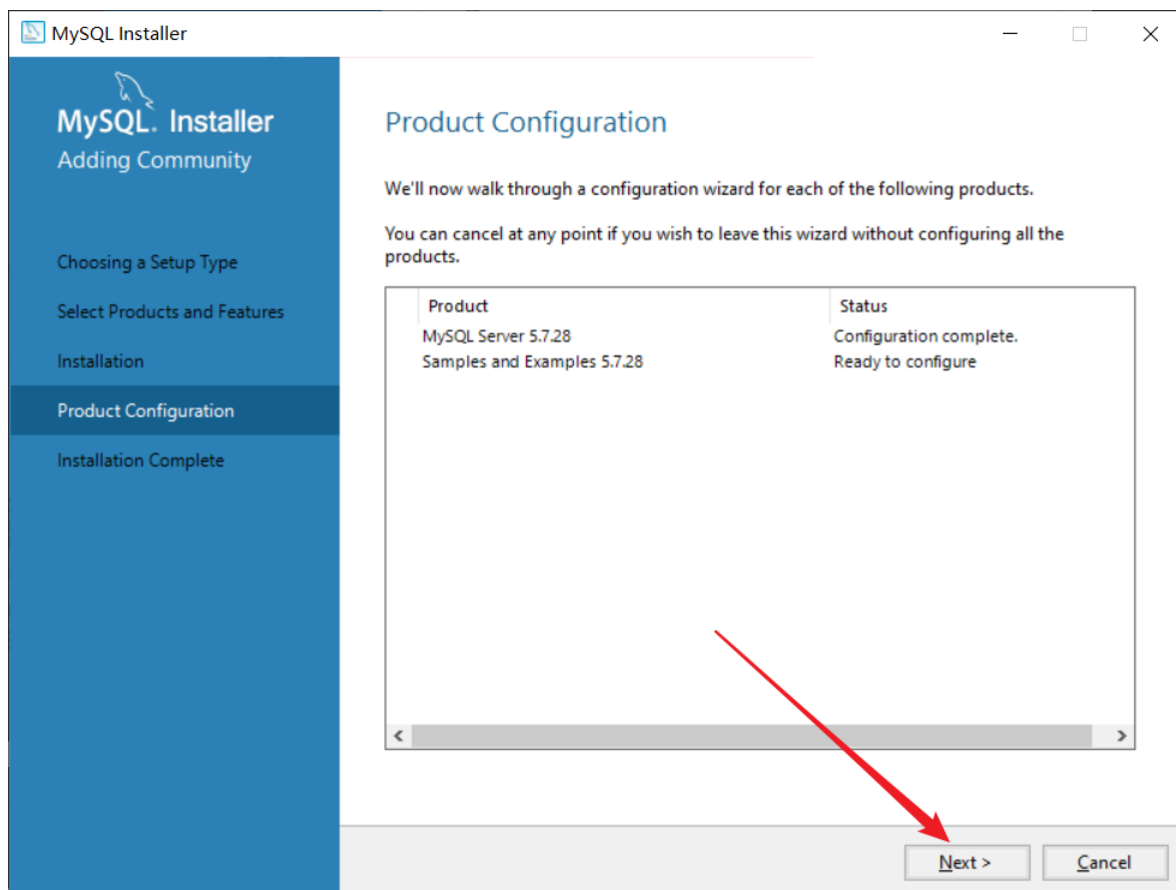
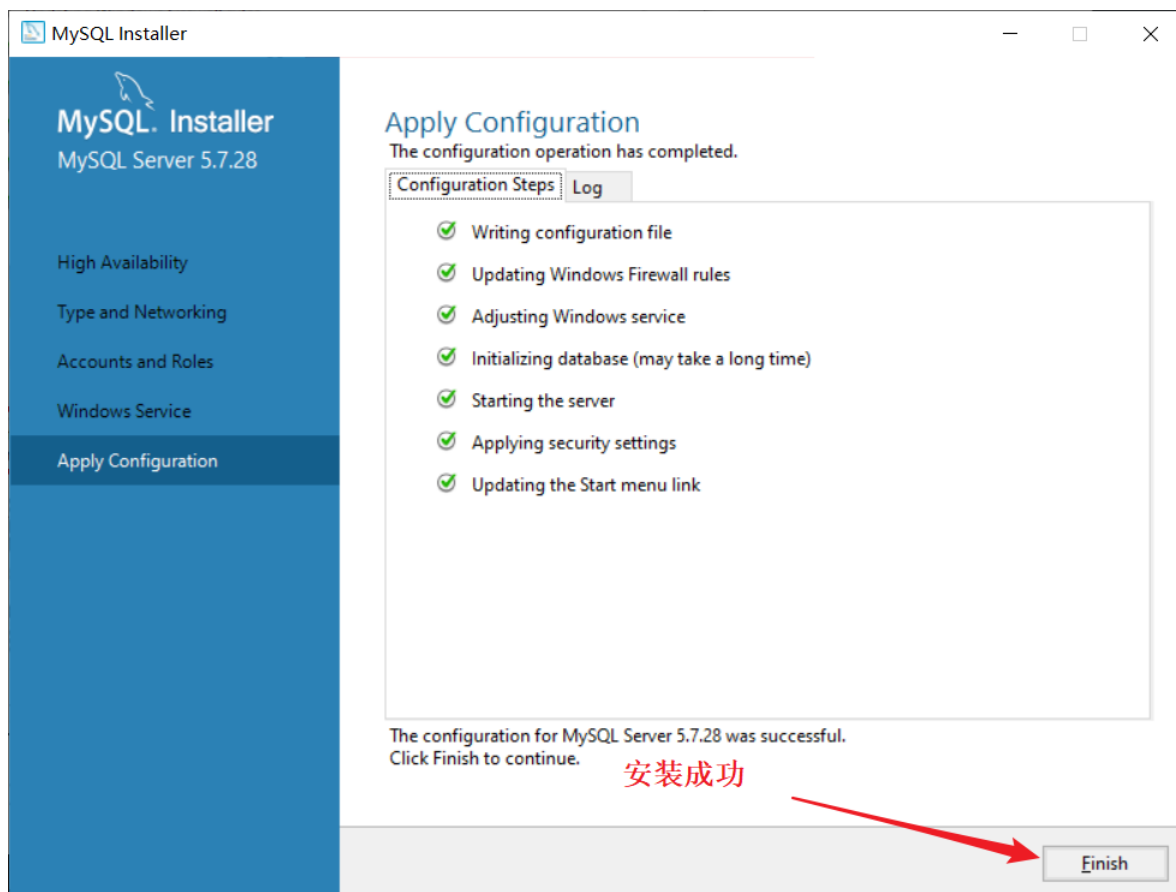
Product	Status
MySQL Server 5.7.28	Ready to configure
Samples and Examples 5.7.28	Ready to configure

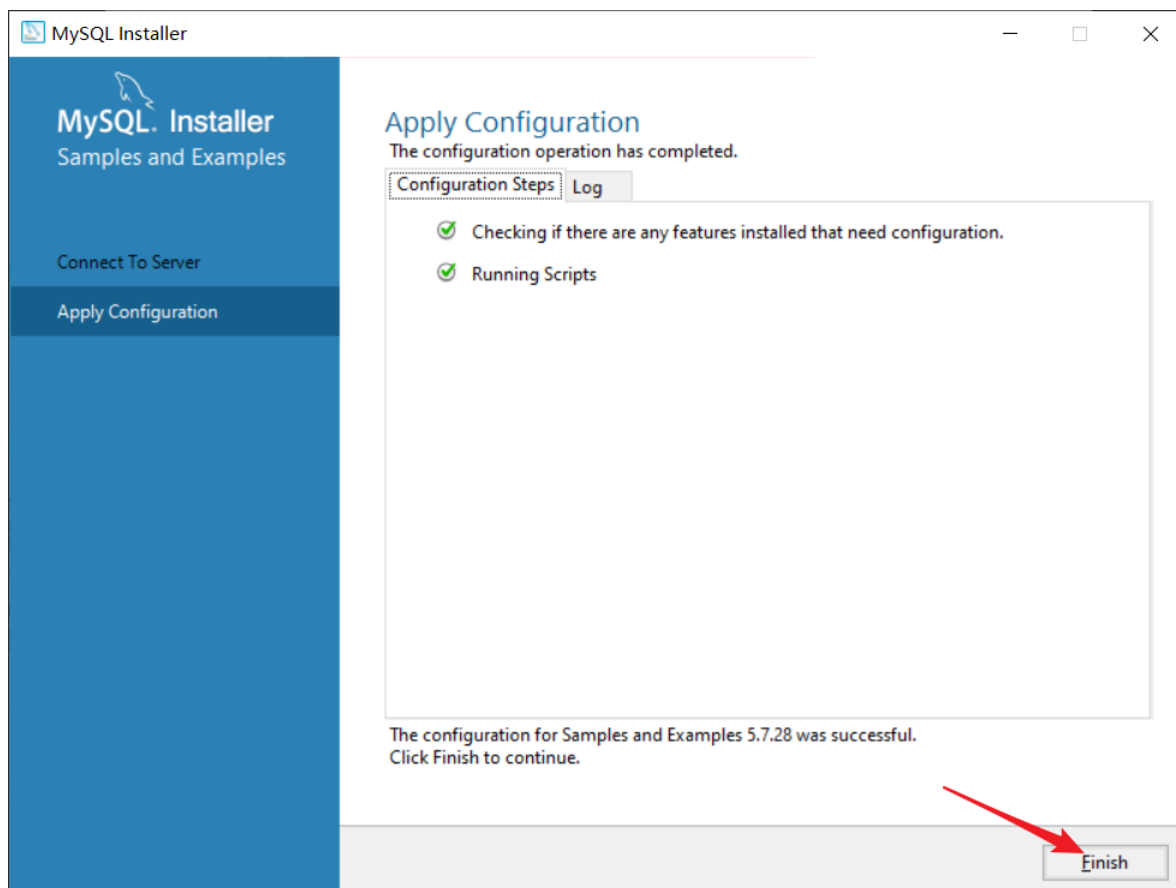
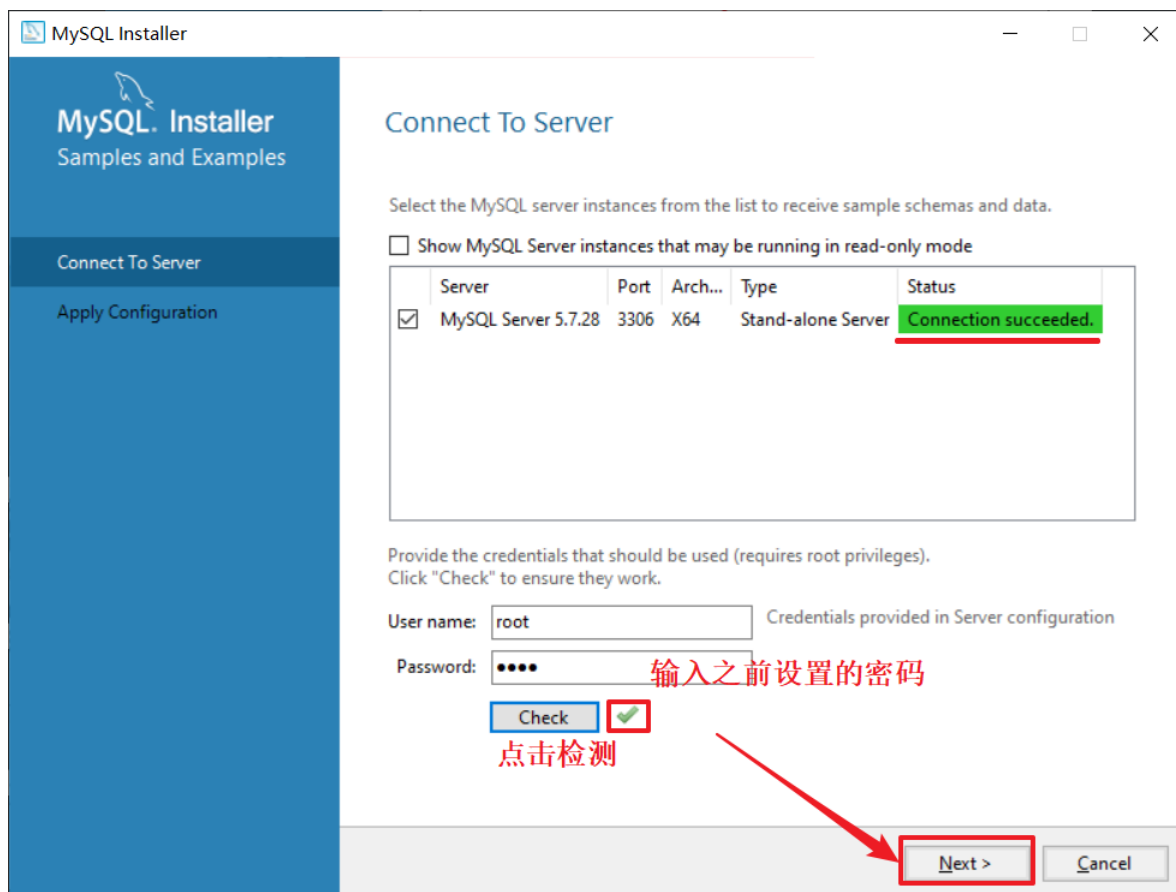
Next >

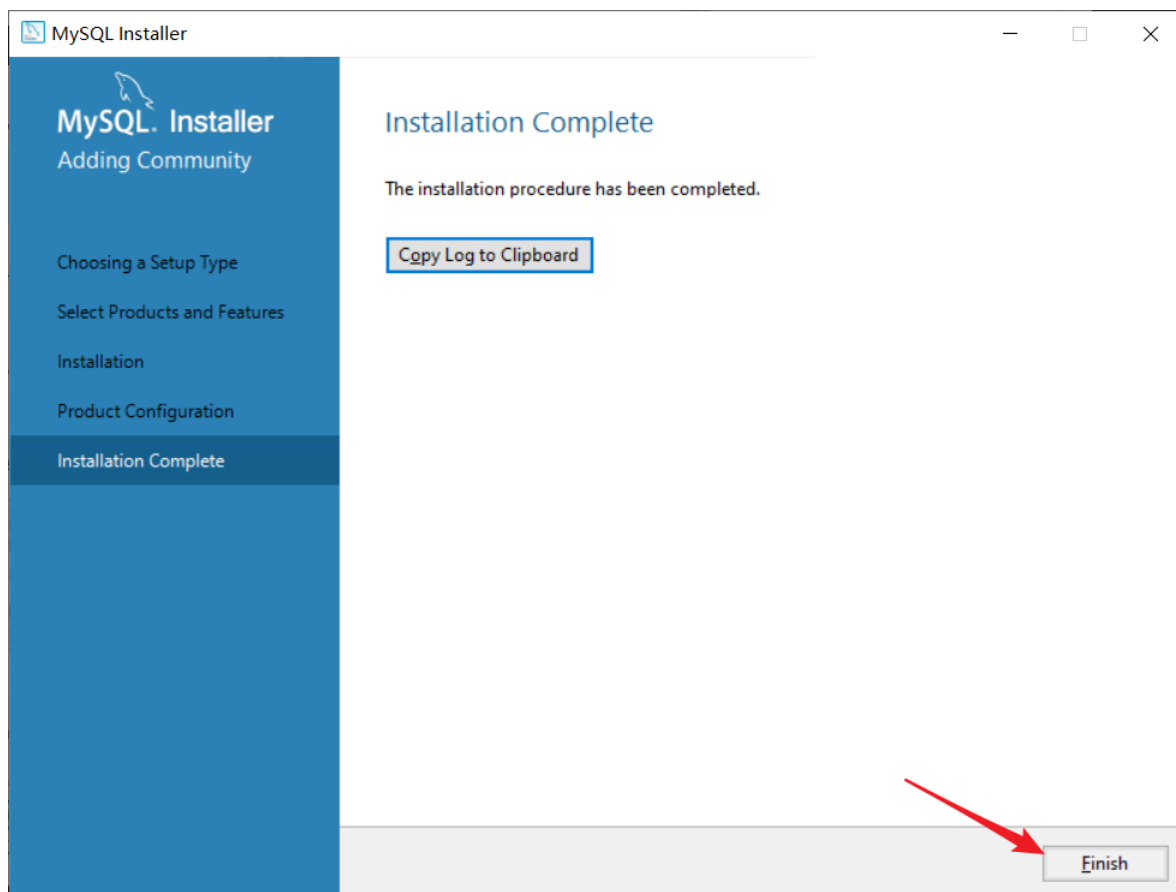
Cancel











知识点2 mysql的命令行客户端的使用

常用指令	含义
net start mysql	启动mysql服务
mysql -u root -p	使用root账户登录，按enter之后需输入密码
show databases;	查看所有数据库
create database 数据库名;	创建数据库
use 数据库名;	使用数据库
show tables;	查看某数据库所有表
create table 表名(字段1 类型1,字段2 类型2...);	创建表
select * from 表名;	查询表
insert into 表名(字段1,字段2) values(值,值2);	插入数据
update 表名 set 字段=值 where 条件;	修改数据
delete from 表名 where 条件;	删除数据
drop table 表名;	删除表
drop database 数据库名;	删除数据库
exit;	退出

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
```

```
6 rows in set (0.00 sec)
```

```
mysql> create database test1;  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sakila |  
| sys |  
| test1 |  
| world |  
+-----+  
7 rows in set (0.00 sec)
```

```
mysql> use test1;  
Database changed  
mysql> show tables;  
Empty set (0.00 sec)
```

```
mysql> create table stu(  
    -> id int not null,  
    -> name varchar(20) not null  
    -> );  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> show tables;  
+-----+  
| Tables_in_test1 |  
+-----+  
| stu |  
+-----+  
1 row in set (0.00 sec)
```

```
mysql> insert into stu(id,name) values(1,'yu');  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from stu1;  
ERROR 1146 (42S02): Table 'test1.stu1' doesn't exist  
mysql> select * from stu;  
+----+-----+  
| id | name |  
+----+-----+  
| 1 | yu |  
+----+-----+  
1 row in set (0.00 sec)
```

```
mysql> update stu set name='zhang' where id=1;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1 Changed: 1 warnings: 0
```

```
mysql> delete from stu where id=1;  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from stu;
Empty set (0.00 sec)

mysql> drop table stu;
Query OK, 0 rows affected (0.01 sec)

mysql> show tables;
Empty set (0.00 sec)

mysql> drop database test1;
Query OK, 0 rows affected (0.00 sec)
```

知识点3 数据库的基本概念



MySQL数据库

MySQL是一个关系型数据库管理系统，由瑞典MySQL AB公司开发，目前属于Oracle公司。

关系型数据库

把所有的数据都通过表格的形式存储起来，表格分为行和列，每行都称为一条记录，每列称为数据域，许多行和列就组成了一张表。如下，该表格中有一个行数据，姓名和年龄就是该表的列

姓名	年龄
张三	20

表和表之间建立主外键来进行关联，多个表就组件了关系型数据库

特点

- 1、MySQL是开源的，所以你不需支付额外的费用
- 2、MySQL支持大型数据库，支持5000万条记录的数据仓库
- 3、MySQL使用标准的SQL数据语言形式。
- 4、MySQL可以运行于多个系统上，并且支持多种语言。这些编程语言包括C++、Python、Java、PHP等等

知识点4 关系型数据库和非关系型数据库的区别

关系型数据库和非关系型数据库的区别

关系型数据库：

通过主外键关联来建立表与表之间的关系

非关系型数据库：

数据以对象的形式存储在数据库中，而对象之间的关系通过每个对象自身的属性来决定

关系型数据库中，查询一条数据结果是一个对象;非关系型数据库中，我们查询一条数据，结果出来一个数组

比如有一个员工的数据：

姓名：张三，性别：男，员工号：E001，部门：开发部

还有一个部门的数据：

部门：开发部，部长：李四

关系型数据库中，我们创建员工表和部门表来存这两条数据，并且员工表中的部门存储的是部门表中的主键

非关系型数据库中，我们创建两个对象，一个是员工对象，一个是部门对象，用Java来表示就是

```
class Employee{
    int id;
    String name;
    String sex;
    String no;
    int deptId;
}

class Department{
    int id;
    String deptName;
    String deptBoss;
}
```

通过设置Employee类的deptId等于Department类的id来建立这种关系

数据库类型	特性	优点	缺点
关系型数据库	1、关系型数据库，指的就是二维表及其之间的联系所组成的一个数据组织 2、关系型数据库的最大特点就是事务的一致性	1、容易理解 二维表结构是非常贴近逻辑世界一个概念 2、使用方便 通用的SQL语言使得操作关系型数据库非常方便 3、易于维护 丰富的完整性大大降低了数据冗余和数据不一致的概率	1、为了维护一致性所付出的巨大代价就是其读写性能比较差 2、固定的表结构 3、高并发读写需求 4、海量数据的高效率读写
非关系型数据库	1、使用键值对存储数据； 2、分布式； 3、一般不支持事务特性； 4、非关系型数据库严格上不是一种数据库，应该是一种数据结构化存储方法的集合。	1、无需经过sql层的解析，读写性能很高 2、基于键值对，数据没有耦合性，容易扩展	1、不提供sql支持，学习和使用成本较高 2、无事务处理

第2章 MYSQL存储引擎和数据类型

知识点1 MySQL的存储引擎



MySQL数据库存储引擎

在关系型数据库中，数据存储在表中，表由行和列组成。

开发中，可能需要各种不同的表，有的表简单，有的表复杂，有的读取快，有的更新快等根据对数据的处理需求，使用不同的存储引擎，可以将mysql数据库的性能发挥最大

查询mysql数据库中的存储引擎：show engines

存储引擎	描述
InnoDB	事务型数据库引擎，支持事务，支持行级锁定，MySQL 5.5 起成为默认数据库引擎
MyISAM	MySQL 5.0 之前的默认数据库引擎，最为常用。拥有较高的插入，查询速度，但不支持事务
Memory	所有数据置于内存的存储引擎，拥有极高的插入，更新和查询效率。但是会占用和数据量成正比的内存空间。并且其内容会在 MySQL 重新启动时丢失
Merge	将一定数量的 MyISAM 表联合而成一个整体，在超大规模数据存储时很有用
Federated	将不同的 MySQL 服务器联合起来，逻辑上组成一个完整的数据库。非常适合分布式应用
Archive	非常适合存储大量的独立的，作为历史记录的数据。因为它们不经常被读取。Archive 拥有高效的插入速度，但其对查询的支持相对较差

若要修改存储引擎：`%mysql_home%\data\my.ini` 找到[mysqld] 在下面添加：`default-storage-engine=存储引擎;`

```
/*
```

```
存储引擎
```

```
一般：myisam和innodb，实际上是对表的操作（也就是索引引擎是针对表的，不是针对数据库的）
```

```
会看到这三种的文件后缀：
```

```
*/
```

```
columns_priv.frm
```

```
columns_priv.MYD
```

```
columns_priv.MYI
```

```
help_topic.frm
```

```
help_topic.ibd
```



```
/*  
上面一个是myisam索引引擎的文件后缀，下面一个是innodb的  
.frm就是表数据文件，两种结构的索引引擎都一样这块  
而.MYD .MYI就是MYISAM索引引擎的索引数据结构文件  
*/
```

知识点2 MySQL的数据类型

MySQL数据库常用数据类型

日期	用法
date	'2018-08-08'
datetime	'2018-08-08 20:08:08'
timestamp	和datetime相似

数值	说明
int	整型
float	单精度浮点型
double	双精度浮点型
decimal(有效位数,精度)	浮点型

字符串	说明
char(n)	固定长度字符串
varchar(n)	可变成都字符串

总体来说，常用数据类型和SQL Server数据库一样

知识点3 MySQL的数据属性

MySQL数据库常用数据属性

属性	描述
primary key	主键、标识唯一实体
auto_increment	标识列，从1开始每次自增1
unique	唯一
default	默认，字符串类型需要加上单引号
not null	不为空
null	为空
foreign key	外键，foreign key(字段) references 主表(字段)
check	检查约束，设置check约束无效，mysql官方没有实现该功能，若要解决，可使用触发器

除了标识列，其余均和SQL Server数据库一样

第3章 MYSQL数据库基础

知识点1 数据库的增加和删除操作

MySQL数据库操作

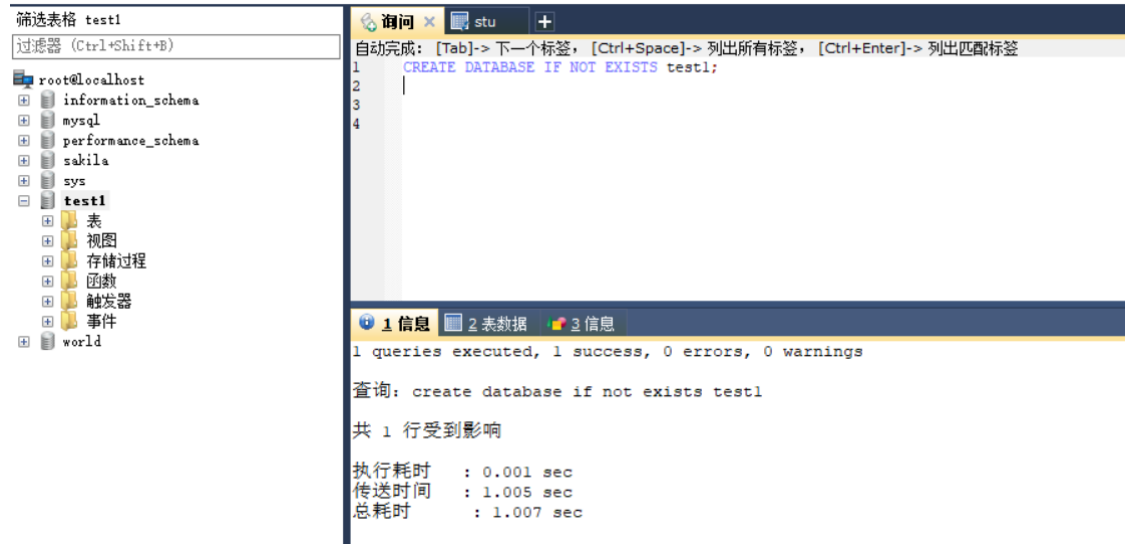
指令			说明
create	database	数据库名;	创建数据库
drop	database	数据库名;	删除数据库
use	数据库名;		使用某数据库
show	databases;		查看当前所有的数据库

- 1、创建时使用下列语句，当数据库不存在时才创建

create database if not exists 数据库名;

2、删除时使用下列语句，当数据库存在时才删除

drop database if exists 数据库名;



```

root@localhost
├─ information_schema
├─ mysql
├─ performance_schema
├─ sakila
├─ sys
└─ world

```

```

1 CREATE DATABASE IF NOT EXISTS test1;
2
3
4 DROP DATABASE IF EXISTS test1;
5
6

```

1 信息 2 表数据 3 信息

1 queries executed, 1 success, 0 errors, 0 warnings

查询: drop database if exists test1

共 0 行受到影响

执行耗时 : 0.001 sec
 传送时间 : 1.004 sec
 总耗时 : 1.006 sec

知识点2 数据表的操作

MySQL数据库表操作

关系型数据库，是把数据存储在表中
表由行和列组成，每列称为字段，每行称为一条记录

创建表

```

create table 表名
(
    字段1 数据类型 属性,
    字段2 数据类型 属性
)

```

数据类型

int、float、double、date、datetime、char、varchar.....

属性

primary key、unique、auto_increment、null、not null.....

指令

show tables; --查看某数据库中所有表
 describe 表名; --查看某表的结构

```
DESCRIBE stu; #查看表结构
```

1 结果 2 信息 3 表数据 4 信息						
(只读)						
Field	Type	Null	Key	Default	Extra	
id	int...	7B NO	PRI	(NULL)	OK	auto_increment
name	var...	11B NO		(NULL)	OK	
sex	cha...	7B NO		(NULL)	OK	
birthday	date	4B NO		(NULL)	OK	

```

CREATE DATABASE IF NOT EXISTS test1;

USE test1;

CREATE TABLE stu(
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    NAME VARCHAR(20) NOT NULL,
    sex CHAR(2) NOT NULL,
    birthday DATE NOT NULL
);

SELECT * FROM stu; #查询表数据
DESCRIBE stu; #查看表结构

```

删除表

```

drop table 表名;
drop table if exists 表名; --当表存在时才删除

```

复制表

```

create table 表1 select * from 表2; --把表2数据复制到表1, 不能复制主键、唯一键
create table 表1 like 表2; --把表2结构复制到表1, 不能复制数据

```

修改表

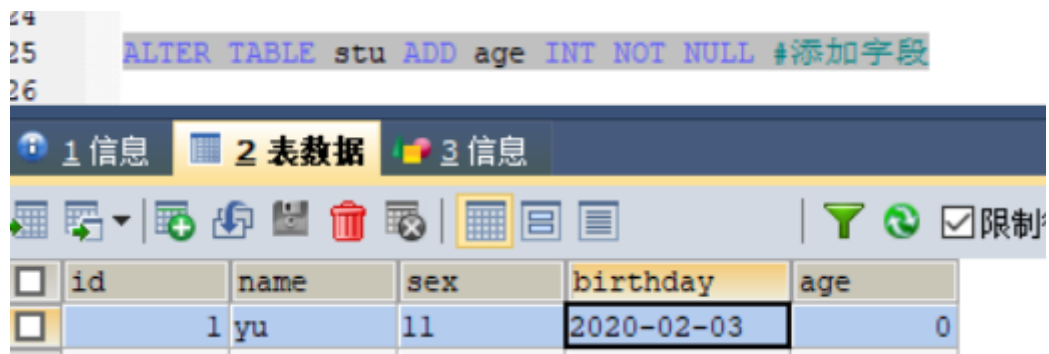
指令	说明
alter table 表名 add 字段名 数据类型 属性	向表添加新字段
alter table 表名 drop 字段名	删除表中某字段
alter table 表名 change 字段名 新字段名 数据类型 属性	修改表中某字段
alter table 表名 add primary key (字段名)	把表中某字段设置为主键
alter table 表名 rename to 新表名	修改表名

```

CREATE TABLE s1 SELECT * FROM stu; #把stu表复制到s1, 不能复制主键、唯一键
SELECT * FROM s1;
DESCRIBE s1;
DESCRIBE stu;

CREATE TABLE s2 LIKE stu; #把stu把结构复制到s2表中, 但是不能复制数据
SELECT * FROM s2;
DESCRIBE s2;

```



```
ALTER TABLE stu DROP age #删除字段
```

1 信息 2 表数据 3 信息			
id	name	sex	birthday
1	yu	11	2020-02-03
(Auto)	(NULL)	(NULL)	(NULL)

```
27 ALTER TABLE stu CHANGE sex age INT NOT NULL; #修改字段
```

```
28 #
```

1 信息 2 表数据 3 信息			
id	name	age	birthday
1	yu	11	2020-02-03

```
ALTER TABLE stu ADD age INT NOT NULL #添加字段
ALTER TABLE stu DROP age #删除字段
ALTER TABLE stu CHANGE sex age INT NOT NULL; #修改字段
ALTER TABLE s1 ADD PRIMARY KEY(id); #字段设置主键
DESCRIBE stu;
ALTER TABLE stu1 RENAME TO stu; #修改表名
```

知识点3 数据的添加操作

MySQL数据库表数据操作

添加数据

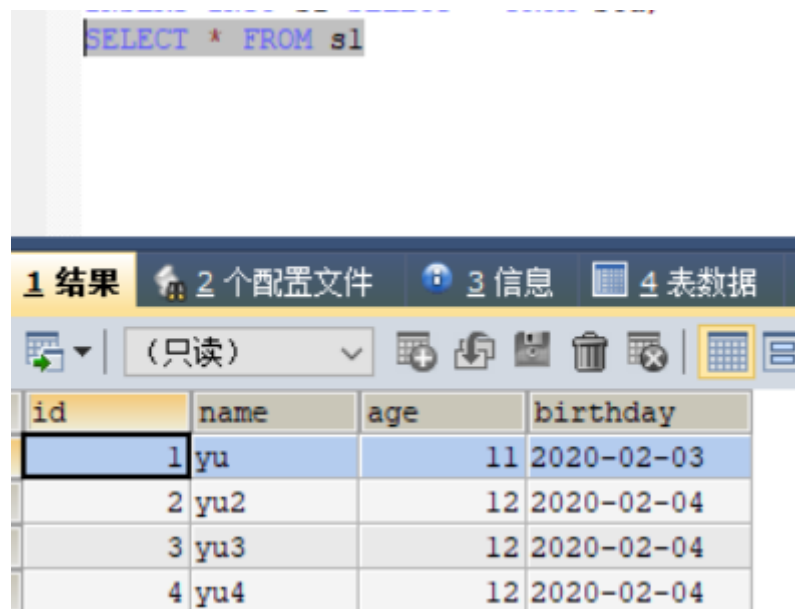
```
insert into 表名[字段1,字段2..] values(值1,值2..);
--当表名后写上字段名时, 值的顺序要和指定字段顺序一致
--当表名后没有字段名时, 值的顺序要和表定义的顺序一致
--对于标识列赋值
(1) 表名后跟上标识列字段, 赋值为0或null
(2) 表名后不指定标识列字段
--对于有默认值的列赋值
(1) 赋值为default关键
(2) 不显示指定默认值列
```

批量添加数据

```
insert into 表1 select */字段 from 表2;
--该方式把表2查询的数据添加到表1中
--前提是表1拥有和表2一样的表结构(create table 表1 like 表2)
```

```
/*
添加数据
*/
#单个添加
INSERT INTO stu(id,NAME,age,birthday) VALUES(2,'yu2',12,'2020-02-04');
INSERT INTO stu(id,NAME,age,birthday) VALUES(3,'yu3',12,'2020-02-04');
INSERT INTO stu VALUES(4,'yu4',12,'2020-02-04');

#批量添加
CREATE TABLE s1 LIKE stu;
INSERT INTO s1 SELECT * FROM stu;
SELECT * FROM s1
```



知识点4 数据的修改操作

MySQL数据库表数据操作

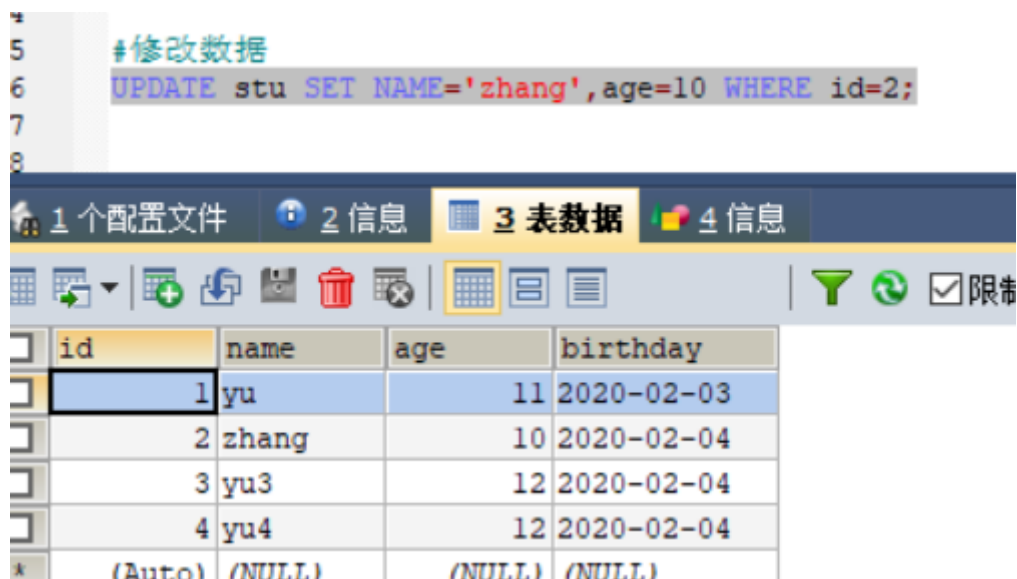
修改数据

update 表名 set 字段=值,字段2=值2...where 条件
 —没有where条件判断时，表示修改表中所有数据

删除数据

delete from 表名 where 条件
 —没有where条件判断时，表示删除表中所有数据
 delete from 表名和truncate table 表名：
 1、都能删除表中的所有数据
 2、delete不会释放表空间，truncate会释放表空间
 3、truncate删除速度快
 4、对于自增列，delete删除后会继续计数，truncate从头开始计数

```
UPDATE stu SET NAME='zhang',age=10 WHERE id=2;
```



知识点5 数据的删除操作

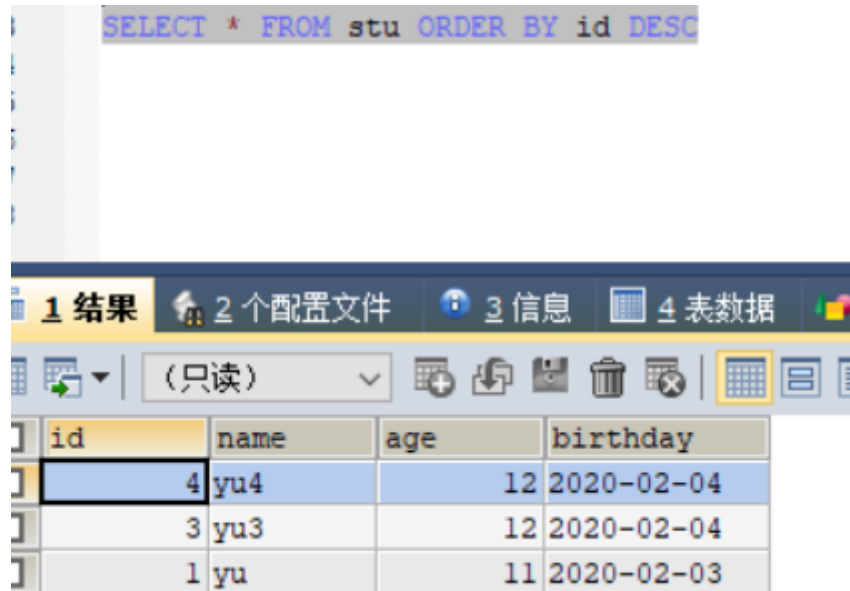
```
DELETE FROM stu WHERE age=10;
```


知识点6 数据的查询操作

查询数据

```
select */字段 from 表名 [where 条件] [order by 排序] [group by 分组]
--*表示所有字段
--where、order by、group by可选
```

```
SELECT * FROM stu WHERE NAME='yu'
SELECT * FROM stu ORDER BY id DESC
```



id	name	age	birthday
4	yu4	12	2020-02-04
3	yu3	12	2020-02-04
1	yu	11	2020-02-03

知识点7 数据高级查询操作（上）

连接查询

将两个或多个表中的数据组合成单个结果集的一种方法

mysql中连接查询包括:

交叉连接、内连接、左连接、右连接

交叉连接

```
select */字段 from 表1,表2... [where 条件]
--若没有where条件关联，得到结果集是表1 * 表2
```

内连接

```
select */字段 from 表1 inner join 表2 on 条件关联 [where 条件]
--把能够通过条件关联上的数据匹配出来
```

左连接

```
select */字段 from 表1 left join 表2 on 条件关联 [where 条件]
--以左边表为准，左边表数据全部匹配，右边匹配条件关联上的数据
```

右连接

```
select */字段 from 表1 right join 表2 on 条件关联 [where 条件]
--以右边表为准，右边表数据全部匹配，左边匹配条件关联上的数据
```

```

65
66 SELECT * FROM class;
67 SELECT * FROM stu;

```

1 结果 2 结果 3 信息

(只读)

id	name
1	T01
2	To2
3	T03

1 结果 2 结果 3 信息 4 表数据 5 信息

(只读)

id	name	age	birthday	classId
1	yu	11	2020-02-03	1
2	yu2	15	2020-02-06	1
3	yu3	12	2020-02-04	2
4	yu4	12	2020-02-04	4

////////////////////////////////////

////

```

69 #交叉查询
70 SELECT * FROM class,stu;
71 #
72 #
73 #
74 #

```

1 结果 2 个配置文件 3 信息 4 表数据 5 信息

(只读)

☒ 限制行 第一行:

id	name	id	name	age	birthday	classId
1	T01	1	yu	11	2020-02-03	1
2	To2	1	yu	11	2020-02-03	1
3	T03	1	yu	11	2020-02-03	1
1	T01	2	yu2	15	2020-02-06	1
2	To2	2	yu2	15	2020-02-06	1
3	T03	2	yu2	15	2020-02-06	1
1	T01	3	yu3	12	2020-02-04	2
2	To2	3	yu3	12	2020-02-04	2
3	T03	3	yu3	12	2020-02-04	2
1	T01	4	yu4	12	2020-02-04	4
2	To2	4	yu4	12	2020-02-04	4
3	T03	4	yu4	12	2020-02-04	4

```

SELECT * FROM class,stu WHERE class.id=stu.classid;
#内连接

```

1 结果 2 个配置文件 3 信息 4 表数据 5 信息

(只读)

☒ 限制行 第一行:

id	name	id	name	age	birthday	classId
1	T01	1	yu	11	2020-02-03	1
1	T01	2	yu2	15	2020-02-06	1
2	To2	3	yu3	12	2020-02-04	2

////////////////////////////////////

////////////////////////////////////

#内连接

```
SELECT * FROM class INNER JOIN stu ON class.id=stu.classid;
```

#左连接

1 结果

2 个配置文件

3 信息

4 表数据

5 信息

(只读) 限制行 第一行:

id	name	id	name	age	birthday	classId
1	T01	1	yu	11	2020-02-03	1
1	T01	2	yu2	15	2020-02-06	1
2	To2	3	yu3	12	2020-02-04	2

////////////////////////////////////
////////////////////////////////////

#左连接

```
SELECT * FROM class LEFT JOIN stu ON class.id=stu.classid;
```

#右连接

结果

2 个配置文件

3 信息

4 表数据

5 信息

(只读) 限制行 第一行:

	name	id	name	age	birthday	classId
1	T01	1	yu	11	2020-02-03	1
1	T01	2	yu2	15	2020-02-06	1
2	To2	3	yu3	12	2020-02-04	2
3	T03	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

#右连接

```
SELECT * FROM class RIGHT JOIN stu ON class.id=stu.classid;
```

结果

2 个配置文件

3 信息

4 表数据

5 信息

(只读) 限制行 第一行:

d	name	id	name	age	birthday	classId
1	T01	1	yu	11	2020-02-03	1
1	T01	2	yu2	15	2020-02-06	1
2	To2	3	yu3	12	2020-02-04	2
(NULL)	(NULL)	4	yu4	12	2020-02-04	4

////////////////////////////////////
////////////////////////////////////

MySQL数据库表数据连接查询

子查询

在一个select查询语句中包含另一个select语句，称为子查询
通常子查询的语句放在where后面充当条件匹配

```
select * from emp where deptId=(select deptId from dept where deptName='学术部')
--查询学术部的所有员工的信息
--后面子查询中查询的字段必须是一个

select * from emp where deptId in(select deptId from dept where deptName in('学术部','教质部'))
--若子查询的结果集有多个，子查询前面应该使用in关键字

select * from emp where exists (select * from dept where deptName='学术部')
--如子查询有结果集，查询所有员工数据，若子查询没有结果集，则不查询数据
```

对于修改、删除的子查询使用和上面方式一样，只是把select换成delete或update
例如：

```
delete from emp where deptId=(select deptId from dept where deptName='学术部')
--删除部门是学术部的所有员工
```

```
#子查询
#查询班级为T01的所有学生
SELECT * FROM stu WHERE classId=1;
SELECT * FROM stu WHERE classId=(SELECT id FROM class WHERE NAME='T01');
```

1 结果	2 个配置文件	3 信息	4 表数据	5 信息
(只读)				
限制行 第一行: 0				
id	name	age	birthday	classId
1	yu	11	2020-02-03	1
2	yu2	15	2020-02-06	1

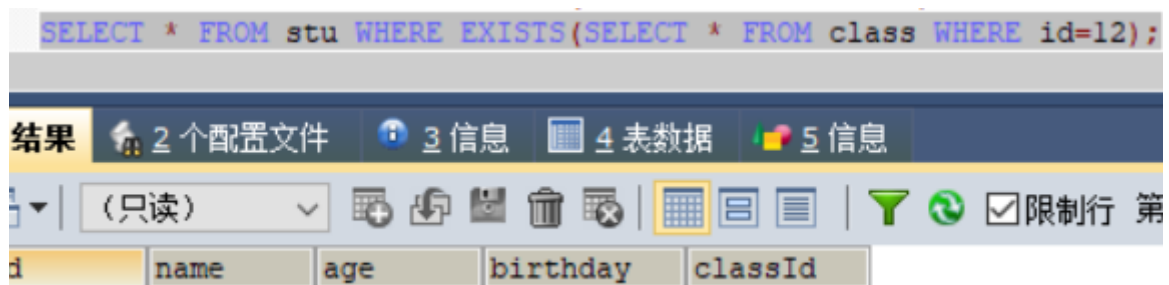
知识点7 数据高级查询操作（下）

```
#查询班级为T01\T02的所有学生
SELECT * FROM stu WHERE classId IN(SELECT id FROM class WHERE NAME IN('T01','T02'));
```

结果	2 个配置文件	3 信息	4 表数据	5 信息
(只读)				
限制行 第一行: 0				
	name	age	birthday	classId
1	yu	11	2020-02-03	1
2	yu2	15	2020-02-06	1
3	yu3	12	2020-02-04	2

```
SELECT * FROM stu WHERE EXISTS(SELECT COUNT(*) FROM class);#后面的有数据，前面的才会有结果
SELECT * FROM stu WHERE EXISTS(SELECT * FROM class);
```

结果	2 个配置文件	3 信息	4 表数据	5 信息
(只读)				
限制行 第一行: 0 行数: 0				
d	name	age	birthday	classId
1	yu	11	2020-02-03	1
2	yu2	15	2020-02-06	1
3	yu3	12	2020-02-04	2
4	yu4	12	2020-02-04	4



```
#把班级为T03的学生删除
DELETE FROM stu WHERE classId=(SELECT id FROM class WHERE NAME='T01');
```

series executed, 1 success, 0 errors, 0 warnings

```
] : delete FROM stu WHERE classId=(SELECT id FROM class WHERE name='T03');
```

1 行受到影响

耗时 : 0.001 sec
 时间 : 0 sec
 时 : 0.002 sec

	id	name	age	birthday	classId
	3	yu3	12	2020-02-04	2
	4	yu4	12	2020-02-04	4
*	(Auto)	(NULL)	(NULL)	(NULL)	(NULL)

知识点8 MySQL中的常用函数

MySQL数据库常用函数

聚合函数	说明
max(字段)	最大值
min(字段)	最小值
avg(字段)	平均值
sum(字段)	和
count(字段)	总数

数学函数	说明
abs(n)	绝对值
floor(n)	向下取整
ceiling(n)	向上取整
round(n,m)	四舍五入，保留m位小数

字符串函数	说明
concat(c,s)	拼接两个字符串
length(s)	字符串长度
substring(s,start,length)	截取，下标从1开始
trim(s)	去首空格
replace(s,old,new)	替换

日期行数	说明
now()	当前系统时间

聚合函数

*/

```
SELECT MAX(age) FROM stu;  
SELECT MIN(age) FROM stu;  
SELECT AVG(age) FROM stu;  
SELECT SUM(age) FROM stu;  
SELECT COUNT(age) FROM stu;
```

/*

数学函数

*/

```
SELECT ABS(-12);  
SELECT FLOOR(3.5); #3 向下取整  
SELECT CEILING(3.5); #4向上取整  
SELECT ROUND(3.14159); #四舍五入
```

/*

字符串

*/

```
SELECT CONCAT('abc','sdf');#abcsdf 拼接  
SELECT LENGTH('sdfdfg'); #6  
SELECT SUBSTRING('asdffgs',2,3)#截取sdf  
SELECT TRIM(' sd');#去首空格  
SELECT REPLACE('asdf','asd','rrr')#替换 rrrf
```

/*

日期

*/

```
SELECT NOW()#获取系统当前时间
```

第4章 MYSQL安全

MySQL数据库权限系统

防止数据有意或无意的被非法查看、修改、删除
围绕两个概念进行：

验证：确定用户是否运行连接数据库
授权：确定用户是否有权限进行操作请求
若果验证不成功，授权就无法进行

MySQL数据库权限控制步骤

- (1) 查看mysql数据库user表中用户是否允许连接
- (2) 若允许连接，查看user表对用户是否授予对所有数据库的全局权限
- (3) 若没有全局权限，查看db表中用户可以和哪些数据库交互
- (4) 若用户在db表中有数据，但主机值为空，查看host表中用户可以和哪些数据库交互，对特定数据库实现特定主机的访问控制
- (5) 若user、db、host表中未有授权，就查看tables_priv和columns_priv表中用户可以和哪些表或列交互

权限控制是从宽泛到特殊的顺序进行检查的

知识点1 创建用户

用户管理

创建用户

```
create user 用户名[@主机名] identified by '密码';  
--主机不指定对所有主机有效、基本所有全局权限都是禁用
```

删除用户

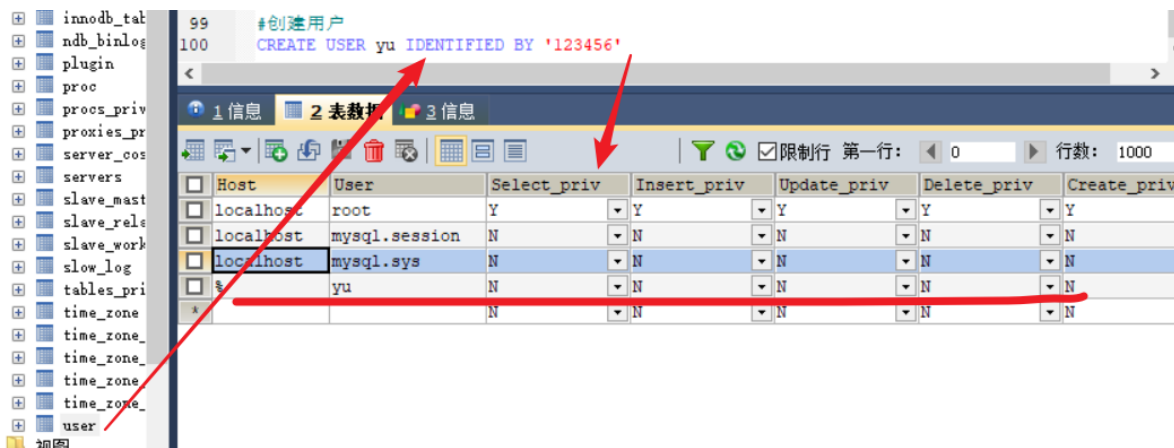
```
drop user 用户名;
```

重命名用户

```
rename user 旧用户名 to 新用户名;
```

修改密码

```
update user set password=password('新密码') where user='用户名';  
--user表在mysql数据库中，需要使用root账号操作
```



知识点2 掌握给用户授权

MySQL数据库用户和权限管理

权限管理

常用权限	说明
all privileges	除授权之外的所有权限
create	创建表权限
drop	删除表权限
alter	修改表权限
insert	添加数据权限
update	修改数据权限
delete	删除数据权限
select	查询数据权限

MySQL数据库用户和权限管理

权限管理

授权

grant 权限 on 表 / 数据库 to 用户名; --多个权限使用逗号分隔开

示例:

```
grant select,insert,delete,update on lesson.* to ceshi
--授予ceshi用户对lesson数据库所有表的增删改查权限
```

```
grant create on lesson to ceshi
--授予ceshi用户对lesson数据库创建表的权限
```

show grants for 用户名; --查看用户拥有的权限

回收

revoke 权限 on 表 / 数据库 from 用户名; --多个权限使用逗号分隔开

注意: 授权和回收权限都需要使用root账户

The screenshot shows a MySQL command line interface with the following commands and their outputs:

```
CREATE USER yu IDENTIFIED BY '123456';
GRANT SELECT, INSERT, UPDATE, DELETE ON root.* TO yu;
```

Below the command line, the MySQL GUI displays the '2 表数据' (Table Data) tab for the 'root' database. The table shows the privileges granted to the user 'yu'.

Host	Db	User	Select_priv	Insert_priv	Update_priv	Delete_priv
localhost	performance_schema	mysql.session	Y	N	N	N
localhost	sys	mysql.sys	N	N	N	N
%	root	yu	Y	Y	Y	Y
*			N	N	N	N

知识点3 撤回用户权限

知识点4 删除用户

用户管理

创建用户

create user 用户名[@主机名] identified by '密码';
--主机不指定对所有主机有效、基本所有全局权限都是禁用

删除用户

drop user 用户名;

重命名用户

rename user 旧用户名 to 新用户名;

修改密码

update user set password=password('新密码') where user='用户名';
--user表在mysql数据库中, 需要使用root账号操作

DROP USER yu;

知识点5 限制访问资源

MySQL数据库限制用户资源

限制每个用户的mysql资源使用，这些限制和其他权限一样，也是通过权限表管理。有四种权限与资源使用有关，都在mysql数据库user表中。

max.connections 用户每小时连接数据库的最大次数
max_questions 用每小时可执行的最大查询次数（使用select的次数）
max_updates 用户每小时可以执行的最大更新次数（使用insert、update的次数）
max_user_connections 用户可维护的最大并发连接数

max_questions	max_updates	max_connections	max_user_connections
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

知识点6 MySQL索引操作

MySQL数据库索引

索引可以提升查询效率

普通索引

最基本的索引，它没有任何限制

create index 索引名 on 表名(字段); --给表中某字段设置索引
drop index 索引名 on 表名; --删除所有

唯一索引

唯一索引列的值必须唯一，但允许有空值。只能有一个空值。

create unique index 索引名 on 表名(字段);

注意：索引创建完成后，在查询数据时where后面若使用到索引列，系统会自动使用索引。

使用ALTER 命令添加和删除索引

有四种方式来添加数据表的索引：

ALTER TABLE tbl_name ADD PRIMARY KEY (column_list); 该语句添加一个主键，这意味着索引值必须是唯一的，且不能为NULL。

ALTER TABLE tbl_name ADD UNIQUE index_name (column_list); 这条语句创建索引的值必须是唯一的（除了NULL外，NULL可能会出现多次）。

ALTER TABLE tbl_name ADD INDEX index_name (column_list); 添加普通索引，索引值可出现多次。

ALTER TABLE tbl_name ADD FULLTEXT index_name (column_list); 该语句指定了索引为FULLTEXT，用于全文索引。

以下实例为在表中添加索引。

```
mysql> ALTER TABLE testalter_tbl ADD INDEX (c);
```

你还可以在 ALTER 命令中使用 DROP 子句来删除索引。尝试以下实例删除索引：


```
mysql> ALTER TABLE testalter_tbl DROP INDEX c;
```

使用 **ALTER** 命令添加和删除主键

主键只能作用于一个列上，添加主键索引时，你需要确保该主键默认不为空（**NOT NULL**）。实例如下：

```
mysql> ALTER TABLE testalter_tbl MODIFY i INT NOT NULL;
```

```
mysql> ALTER TABLE testalter_tbl ADD PRIMARY KEY (i);
```

你也可以使用 **ALTER** 命令删除主键：

```
mysql> ALTER TABLE testalter_tbl DROP PRIMARY KEY;
```

删除主键时只需指定**PRIMARY KEY**，但在删除索引时，你必须知道索引名。

显示索引信息

你可以使用 **SHOW INDEX** 命令来列出表中的相关的索引信息。可以通过添加 **\G** 来格式化输出信息。

尝试以下实例：

```
mysql> SHOW INDEX FROM table_name; \G
```

索引优劣势：

索引优劣势：

索引——**index**：帮助mysql高效获取数据的数据结构

因此，问到你索引是什么的时候，你们要回答，这是一种数据结构

使用索引的优势是：**1**、检索速度快；**2**、排序快

劣势是：**1**、增删改的时候更新表的速度慢，因为表中保存有增加索引的字段，该表的时候数据会同步跟新索引字段**2**、索引本身也是表，被引用字段指向表中的记录，索引要占用空间

索引使用场景：

使用场景

- 1，主键自动建立唯一索引
- 2，频繁作为查询条件的字段应该创建索引
- 3，查询中其他表关联的字段，组合索引性价比更高
- 4，查询排序的字段，排序字段若通过索引去访问将大大提高排序速度
- 5，查询统计或者查询组字段（分组——**group by**）
- 6，单键/组合索引的选择问题，组合索引性价比更高

不建议使用场景

- 1，表记录太少
- 2，经常增删改的表或者字段
- 3，**where**条件里用不到的字段不创建索引
- 4，过滤性不好的不适合创建索引字段

过滤性：比如**sex**这个字段只有男和女

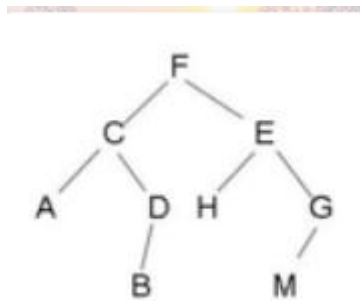
一张表有很多的行数，每行**sex**都是男和女，于是我们只能过滤男和女，无法确定行数，这叫过滤性不好
如果这个字段有唯一性，能够通过该字段的值锁定一行，就叫过滤性好，比如身份证字段

内部原理讲解

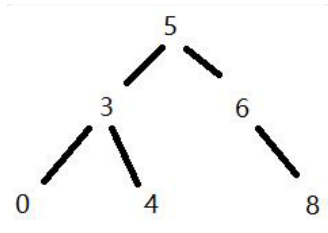
```
select * from user where id=100
```

/*没有索引，执行为链表数据结构，从小到大，直到**id**为**100**的**for**循环*/

/*添加索引后，使用**B+**树 的数据结构，效率极限提升，很小的次数就可以查到**id=100**的行数据，好，这里就出现了**B+**树的结构，我们要讲清楚这个结构就要搞懂**B+**树的来龙去脉，以前给大家提到过一个叫做二叉树的数据结构，也称为查找二叉树，*/



- 1, 左子树不为空, 则左子树上的节点的值小于根节点
- 2, 右子树不为空, 则右子树上的节点的值大于根节点
- 3, 子树遵循以上两点



/*

一般来说有4种遍历方式

前序, 中序, 后序, 层级

前序: 根左右

中序: 左根右--034568

后序: 左右根--

层级: 按层从小到大从左往右

这里我们只列出中序的遍历结果, 结果是顺序递增的, 因此, 所有的遍历中中序遍历是最重要的, 也是使用最广泛的, 那么采用树形结构我们要查找4个节点, 会找多少次呢?

*/

/*

4个节点 $2^2=4$ 找2次

【0--31】一共32个数中找任意一个最多找几次呢?

32个节点 $2^5=32$ 找5次

一共是6个数, 因此

$2^2 < 6 < 2^3$ 次, 最多找3次

你们不要去看具体的节点值, 要看节点个数

最坏的结果就是找到最后一层嘛

那么按照这个推论, 一共有n个数, 找几次?

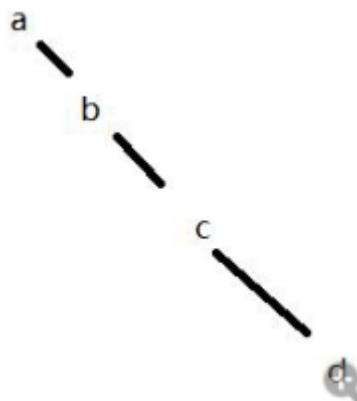
$2^m < n < 2^{m+1} \rightarrow m+1$ 次 $2^{m+1} = n \Rightarrow m+1 = \log_2 n$

看不懂小学数学没有关系, 我们带一带公式算一下21亿个数中, 查找任意一个数需要找几次?

$2^{32} = 21$ 亿, 所以找32次

但是, 问题来了, 其实有一种极端情况没有考虑到

*/



```
/*
```

如果二叉树只有左或者右子树，就是一个单向链表结构，21亿个数还是要找21亿次

我们刚才的那种计算方法，其实是平衡二叉树的计算方法

不能都是左或者右，平衡之后，就可以带入 $\log_2 n$ 这个求次数公式

平衡二叉树又叫做AVL树

AVL树有很多种实现（但是基本的核心是三个（变色，左旋，右旋）

其中的经典方案之一是红黑树

mysql中的索引使用的数据结构其实就是AVL树，但是不是红黑树

我们就要谈一下哪些是使用的红黑树，hashmap

为何hashmap使用红黑树，mysql索引不使用红黑树呢？

这里涉及到一个数据库优化方案的问题，优化一条语句的开销和效率考虑三个地方

cpu memory IO 中央处理器，内存，磁盘读写

数据库在磁盘IO的时候，如果采用红黑树，必然是AVL二叉平衡树的模样，数据量一大，树的层数就多，21亿次就可能有30几行

那么要索引一个节点的时候，在磁盘中要读写30几次才能找到这个节点，有点影响开销，所以不采用红黑树的数据结构

那么为什么hashmap使用的是红黑树呢？

非也非也，其实是hashmap使用的是内存开销，并非磁盘io，hashmap在jvm中开销，jvm在cpu缓存中开销，不涉及到磁盘io，所以快

但是数据库就不一样，操作是基于磁盘的，高并发状态下，大量的数据访问会考验服务器数据库的io性能，由于io速度远远小于cpu缓存和内存读写速度，所以必须要优化磁盘的读写次数，所以数据库不用红黑树结构

为了解决这个多层次问题，所以设计出来了B-树这个结构，后面改良为B+树

```
*/
```

B-树

```
/*
```

那么什么是B树呢？为何后面又改良成了B+树呢？B+树又是怎么回事呢？使用B+树的数据库技术是什么呢？

B+树的索引最佳实践如何呢？

https://blog.csdn.net/qq_26222859/article/details/80631121

<https://www.jianshu.com/p/71700a464e97>

<https://www.cnblogs.com/sujing/p/11110292.html>

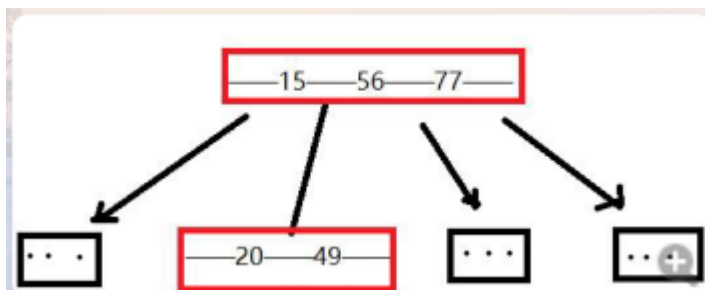
前提是红黑树只适用于内存这种读写速度快，一次IO的硬件中使用，例如hashmap在jvm中使用，jvm在内存中运行

那么对于磁盘IO的情况，改良平衡树就显得尤为重要。

红黑树主要的问题是树的层级随着数据量的增加，会显著增加很多，那么如果能够确保树的层级只有2到3层，就能够显著地提高io效率。

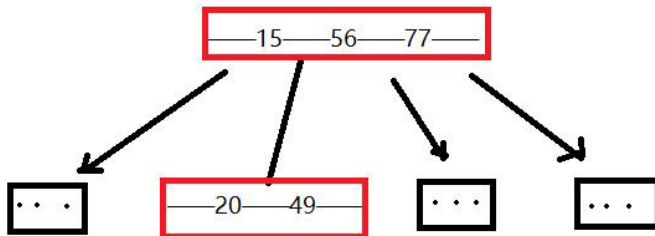
这种总层数在2或者3的树结构是怎么一个样子呢？给大家画一下。

```
*/
```



下面的-20-49-就是根结点-15-56-77-的子

节点

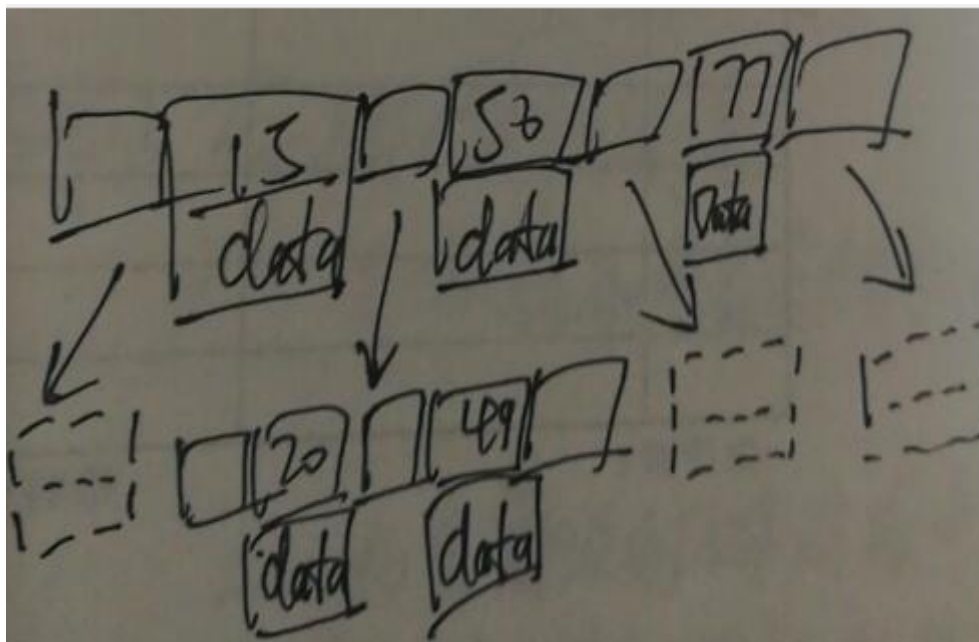


49在15与56之间

从15与56的连接处往下查找

找到—20—49—这个节点

49大于20，往后右走一位，命中查找找到49



data: 就是数据

表里面的每行地址

⇒ (PK)
Col1 Col2 Col3

0x07	15	34	Bob
0x56	18	77	Alice
0x6A	20	5	Jim
0xF3	30	91	Eric
0x90	49	22	Tom
0x77	50	89	Rose

→ My文件.

比如说这张表，col1是索引字段，而每一行之前有一个唯一的行地址，这里面的data就存储的这个行地址0x07 0x56 0x6A，而那个15，56，77，20，49这些数就是索引列中的数据

定义：AVL平衡树的一种实现就是B-树，B-树本身和B-树的每个子树，都是满足二叉平衡树结构的，也就是左小右大

/*

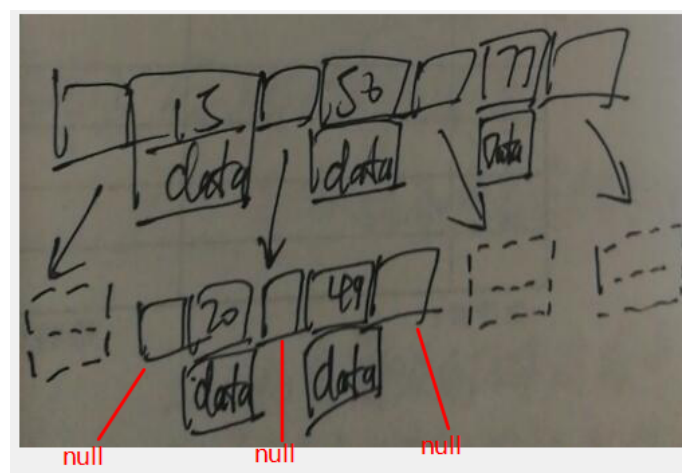
B-树有三个特点：

- 1，叶子节点具有相同的深度，叶子节点的指针为空
- 2，所有索引元素不重复
- 3，节点中的数据索引从左往右递增排列

-20-49-就叫做叶子节点

对于-20-49-这个节点而言，有三个指针，指向下方的三个空元素

*/



/*

为什么mysql最终使用的不是这种B树呢？

也就是说B树有一个致命的缺点，是mysql不能容忍的

如果这种B-树结构中的节点够多，每个节点中的元素又依次从小到大，从左往右依次排列是不是还是一种链表结构

1-2-3-4-5-6-7-...-n-。

那如果有21亿个数，直接gg

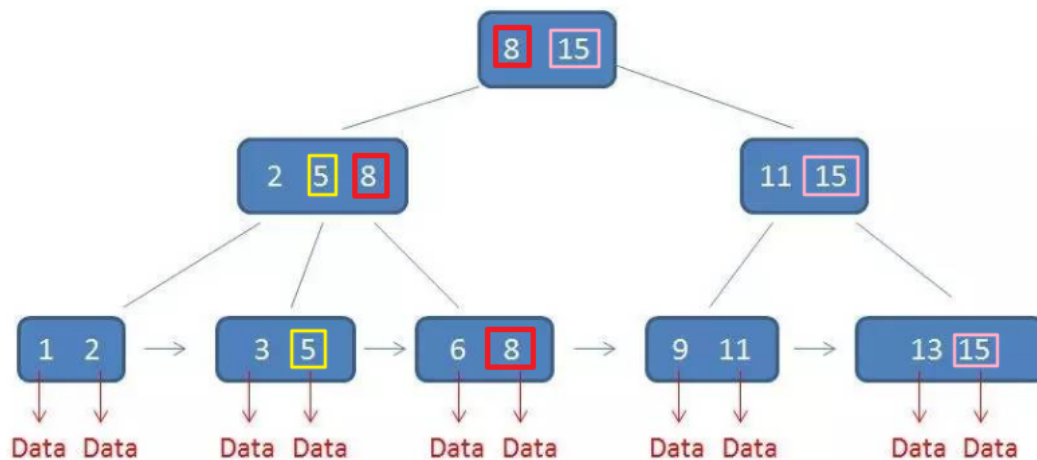
而且这种B-树结构不利于范围查找

所谓范围查找就是

where id>3 and id <100

*/

B+树



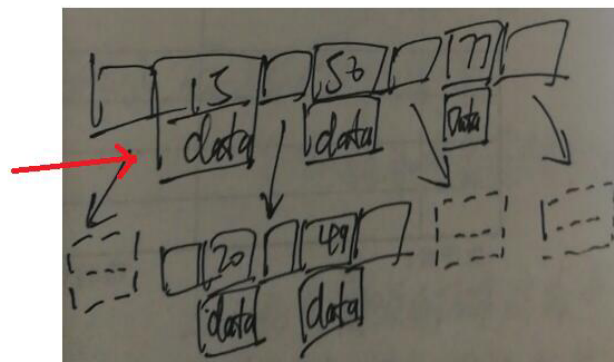
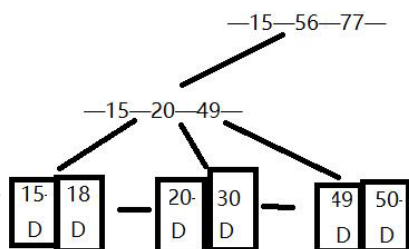
/*

比较‘b-树’我们发现‘b+树’有：

1、会有重复值2、还有一个改动是，叶子节点中才包含完整数据，而非叶子节点比如根结点不包含完整数据

*/

比较‘b-树’，‘b+树’



/*

左面的根结点是没有**data**数据的（表的行地址），右面的每个节点中的索引元素都有**data**数据

左面的叶子节点才有**data**数据（表的行地址）

然后左面的叶子节点是没有子节点的，他们从左往右依次递增，然后15 18 与20 30这个两个节点中有指向指针（双向指针）

‘b+树’：

1. 非叶子节点不存储**data**数据最多有索引值，索引值冗余存在

2. 叶子节点才包含所有索引字段

3. 叶子节点用指针相互连接，提高访问的性能，不用再从根结点每次都从头往下查找

1. 单一节点存储更多的元素，使得查询的IO次数更少。

2. 所有查询都要查找到叶子节点，查询性能稳定。

3. 所有叶子节点形成有序链表，便于范围查询。

*/

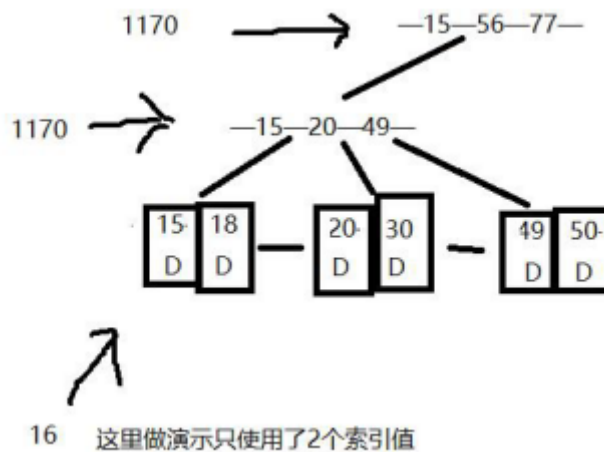
/*

mysql有一套自己的规则生成b+树，我们不用去管他，只是说大家要知道一张表中加了索引，这个索引的数据结构是B+树，B+树中有哪些内容（索引值+表的行地址data）

现在我们来谈一下，如果三层，两层根结点，一层叶子节点，一共存储多少个索引值

非叶子节点是1170个，叶子节点每个可以存储16个索引元素，三层就是 $1170 * 1170 * 16$ 个 = 21902400个2千2万条左右的数据索引

*/



/*

存储引擎

一般：myisam和innodb，实际上是对表的操作（也就是索引引擎是针对表的，不是针对数据库的）

会看到这三种的文件后缀：

*/

columns_priv.frm

columns_priv.MYD

columns_priv.MYI

help_topic.frm

help_topic.ibd

/*

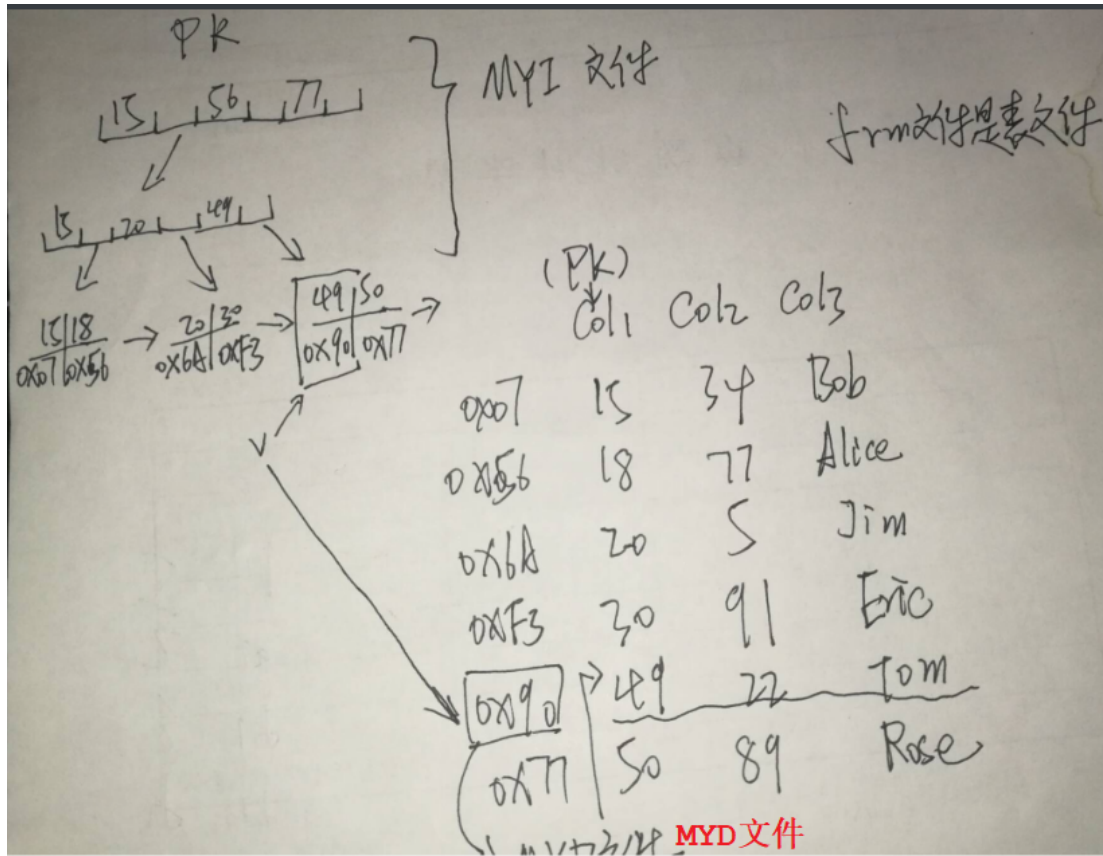
上面一个是myisam索引引擎的文件后缀，下面一个是innodb的

.frm就是表数据文件，两种结构的索引引擎都一样这块

而.MYD .MYI就是MYISAM索引引擎的索引数据结构文件

*/

--- MYISAM索引引擎



/*

然后在myisam索引引擎中会进行上述图形的B+树数据结构化

这一块就是表中索引字段从上往下的值

而其他的非叶子节点中的索引值都是冗余字段，方便查询

每个叶子节点除了索引数值，也对应了表的行首地址

这一块B+树数据结构以及相关数据会被存储在MYI后缀的文件中

然后查询语句查询到49这个索引值时候，在这个MYI后缀的文件中，会根据这个B+树结构中的叶子节点找到49对应的data数据--0x90

通过在0x90对应哪一行的结构存储在后缀是MYD的文件中

然后就从对应的.frm文件中找出对应的行数据

这就是MYISAM索引引擎

*/

MYISAM索引引擎就是用B+树结构存储的索引值和行地址信息

--- INNODB索引引擎

刚才也说了，

- help_topic.frm
- help_topic.ibd

innodb少一个文件

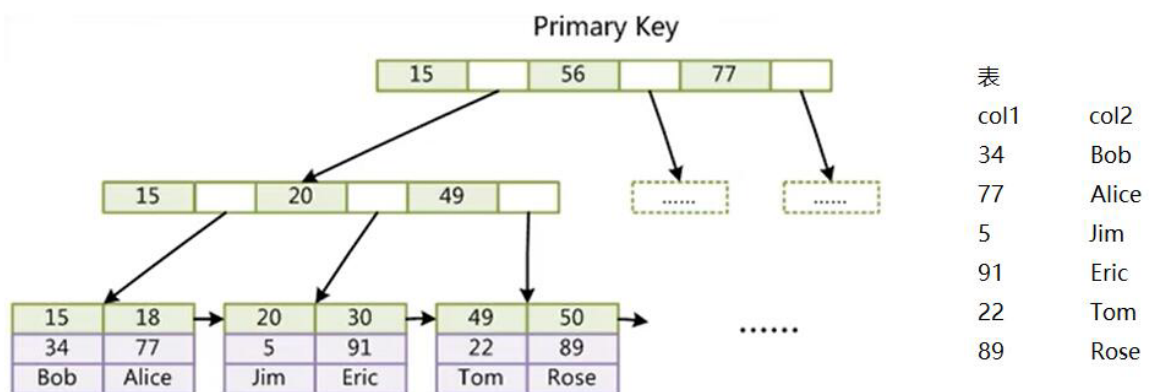
- columns_priv.frm
- columns_priv.MYD
- columns_priv.MYI

/*

也就是说，.ibd是 .MYD和 .MYI文件的功能综合，
一个存储B+树形结构一个用来存储地址指向，
那么innodb一个文件就搞定了两个内容。

在innodb的.ibd后缀的文件中，存储的就是左面的内容

*/

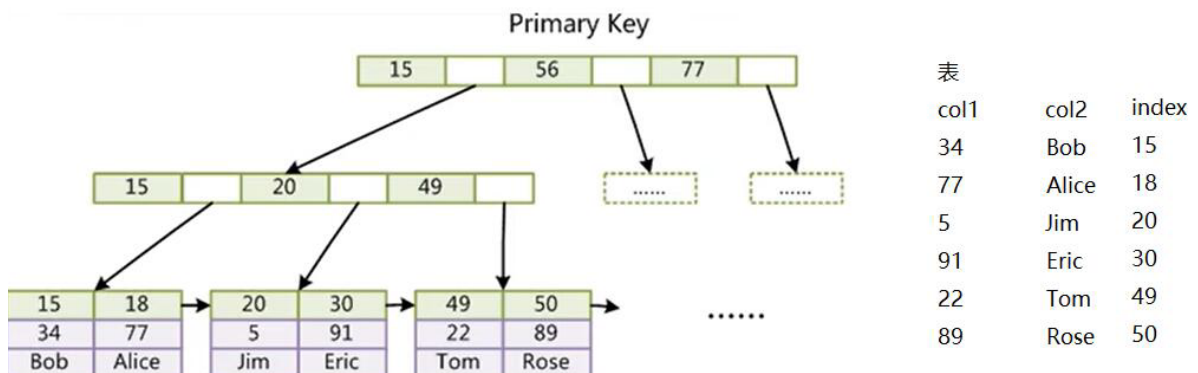


/*

我们会发现15是索引数据，34，Bob是行数据
那么也就是根据索引数据，把对应行的表数据也存储了进来
那么这个15的索引值是哪里来的呢？

由于我们的表只有两列，并没有索引值，因此，innodb索引引擎的机制是，从表中查找有唯一性数据的行，
如果没有就自动帮我们加上一行

*/

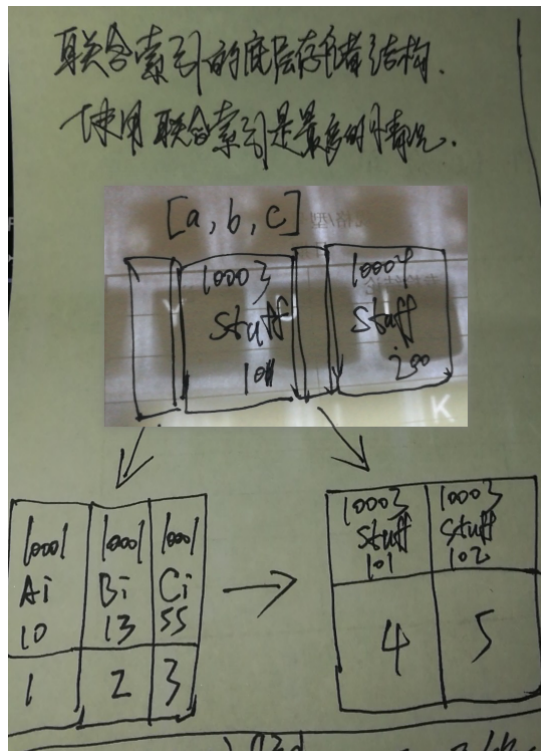


```

CREATE TABLE `employees` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(24) NOT NULL DEFAULT '' COMMENT '姓名',
  `age` int(11) NOT NULL DEFAULT '0' COMMENT '年龄',
  `position` varchar(20) NOT NULL DEFAULT '' COMMENT '职位',
  `hire_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '入职时间',
  PRIMARY KEY (`id`),
  KEY `idx_name_age_position` (`name`,`age`,`position`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8 COMMENT='员工记录表';

INSERT INTO employees(name,age,position,hire_time) VALUES('LiLei',22,'manager',NOW());
INSERT INTO employees(name,age,position,hire_time) VALUES('HanMeimei', 23,'dev',NOW());
INSERT INTO employees(name,age,position,hire_time) VALUES('Lucy',23,'dev',NOW());

```



```
/*
```

联合索引的化就是将索引包含的多个字段，从上到下排列好，然后从上到下从左到右依次比对

比如，数据是10001 Bi 13这条数据和10001 Ai 10比较

由于都是10001所以比对第二条，Ai与Bi，A在字典顺序上先于B

因此Bi那条数据就排第一个叶子节点的第二个索引数据位置

```
*/
```

```
/*
```

索引

```
*/
```

#创建

```
CREATE INDEX index_stu_name ON stu(NAME)
```

#使用，提示系统性能

```
SELECT * FROM stu WHERE NAME='yu3'
```

第5章 视图

知识点1 视图的基本概念

MySQL数据库视图

- 1、视图是包含由一张或多张表的列组成的数据集。
视图中的记录是由一条查询语句执行后所得到的查询结果所构成的。
 - 2、视图是一张虚拟表。
其结构和数据是建立在对表的查询基础上。
 - 3、视图中并不存放数据，数据存放在视图所引用的真实表中。
 - 4、同一张原始表，根据不同用户的不同需求，可以创建不同的视图。
- 优点：
- (1) 简单性，把经常查询的字段的数据封装到视图，避免每次查询重写语句
 - (2) 安全性，通过视图把非敏感数据提供给第三方
 - (3) 可维护性，视图可以简化非常复杂的查询

视图和表的区别

- 1、视图是虚拟表，并不真实存在
- 2、视图中不存放数据，数据存放在真实表中，视图只是去引用真实表中的数据
- 3、视图可以对真实表的列进行筛选，组成新的结果集数据

知识点2 创建视图

视图创建

```
create [or replace] view 视图名  
as  
    查询语句
```

使用

把视图当做普通表使用即可，如：`select * from 视图名 where 条件...`

查看视图字段信息

```
describe 视图名
```

查看视图创建信息

```
show create view 视图名
```

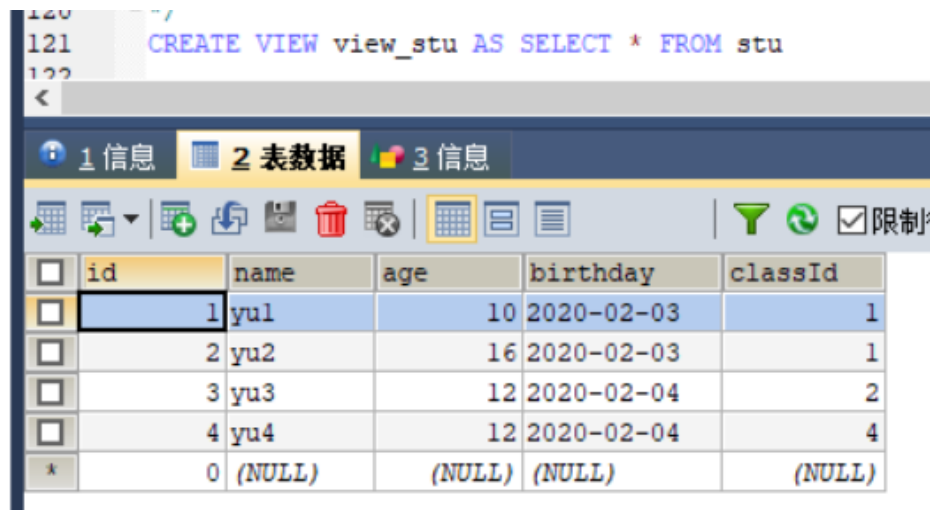
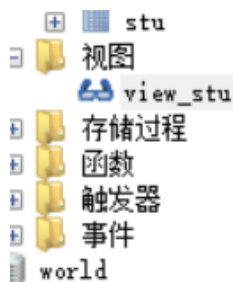
视图修改

```
alter view 视图名  
as  
    新查询语句
```

视图删除

```
drop view 视图名
```

注意：修改视图中的数据，原始表的数据会被更改



知识点3 修改视图



ALTER VIEW view_stu AS SELECT * FROM stu WHERE NAME='yu2'

id	name	age	birthday	classId
2	yu2	16	2020-02-03	1
0	(NULL)	(NULL)	(NULL)	(NULL)

知识点4 使用视图



#使用视图

SELECT * FROM view_stu

d	name	age	birthday	classId
1	yu1	10	2020-02-03	1
2	yu2	16	2020-02-03	1
3	yu3	12	2020-02-04	2
4	yu4	12	2020-02-04	4

SELECT * FROM view_stu WHERE NAME='yu2'

id	name	age	birthday	classId
2	yu2	16	2020-02-03	1

知识点5 删除视图

```
/*
视图
*/
#创建
CREATE VIEW view_stu AS SELECT * FROM stu
#修改视图
ALTER VIEW view_stu AS SELECT * FROM stu WHERE NAME='yu2'
#使用视图
SELECT * FROM view_stu
SELECT * FROM view_stu WHERE NAME='yu2'
#删除视图
DROP VIEW view_stu
```

第6章 存储过程

知识点1 了解存储过程的基础知识

MySQL数据库存储过程

类似编程语言中的函数

存储过程可以带参数，也可以返回结果

存储过程中封装一组sql代码，以完成某种特定的功能

优点

- (1) 一致性，把类似的功能封装为存储过程，减少重复的开发
- (2) 高性能，存储过程执行速度快
- (3) 安全性，存储过程可以确保开发人员只能访问提供的业务所需信息

存储过程的分类

存储过程

存储过程不能有返回值

存储过程可以有多个in,out,inout参数

存储过程可以调用存储函数

存储过程一般是作为一个独立的部分来执行

存储函数

存储函数有且只有一个返回值

存储函数只能有输入参数，而且不能带in

存储函数不能调用存储过程

存储函数一般作为查询语句的一个部分来调用，如：sum()

归纳

存储过程中的语句功能更强大，存储过程可以实现很复杂的业务逻辑

存储函数只完成查询的工作，可接受输入参数并返回一个结果，也就是函数实现的功能针对性比较强。

返回上一级

存储过程

语法

```
create procedure 存储过程名(类型 参数名1 数据类型1,类型 参数名2 数据类型2...)
begin
    逻辑代码
end;
```

参数类型

```
in        --默认的，输入参数
out       --输出参数，可以在执行完存储过程后获取
inout     --输入输出参数
```

调用

```
call 存储过程名(参数);
```

存储函数

语法

```
create function 函数名(参数名 参数类型...)
returns 数据类型
begin
    逻辑代码
    return 和数据类型对应的数据
end;
```

说明

参数只能有输入参数，但是不能显示写in；必须有一个返回值
可以在查询语句中使用函数，像：max(),min()...

知识点2 创建存储例程的方式

MySQL编程 变量

声明

declare 变量名 数据类型;

赋值

set 变量名 = 值;

查询赋值

select 字段1,字段2... into 变量名1,变量名2.. from 表名

输出变量的值

select 变量名 as 别名

存储过程或函数之外声明变量

1、@变量名

没有初始值

2、set @变量名 = 值;

有初始值

完成

- 1、无参数存储过程
- 2、有输入参数的存储过程
- 3、有输出参数的存储过程
- 4、有输入输出参数的存储过程
- 5、无参数存储函数
- 6、有参数存储函数

#无参数存储过程

#使用存储过程查询输出员工人数

```
CREATE PROCEDURE proc_emp_count()
```

```
BEGIN
```

```
    DECLARE c INT;#声明变量，存储人数
```

```
    SELECT COUNT(*) INTO c FROM emp;
```

```
    SELECT c AS 员工总人数;
```

```
END;
```

```
CALL proc_emp_count();
```

#有参数,根据员工id查询出员工姓名

```
CREATE PROCEDURE proc_select_empName_ById(IN id INT )
```

```
BEGIN
```

```
    DECLARE n VARCHAR(20);
```

```
    SET n = (SELECT empName FROM emp WHERE empId=id);
```

```
    SELECT n;
```

```
END;
```

```
CALL proc_select_empName_ById(2);
```

```
SET @id=3;
```

```
CALL proc_select_empName_ById(@id);
```

#有输出参数的

#根据员工id查询出员工姓名

```
CREATE PROCEDURE proc_out_empName_byId(IN id INT,OUT n VARCHAR(20))
```

```
BEGIN
```

```
SET n = (SELECT empName FROM emp WHERE empId=id)
```

```
END;
```

```
CALL proc_out_empName_byId(1,@n);
```

```
SELECT IFNULL(@n, '员工不存在');
```

#无参数

#返回emp员工认数

```
CREATE FUNCTION fun_select_emp_count()
```

```

RETURNS INT
BEGIN
DECLARE c INT;
SELECT COUNT(*) INTO c FROM emp;
END;

SELECT fun_select_emp_count();
SET @c = fun_select_emp_count();
SELECT @cs;

#根据id返回姓名
CREATE FUNCTION fun_return_emp_byId(id INT)
RETURNS VARCHAR(20)
BEGIN
DECLARE n VARCHAR(20);
SELECT empName INTO c FROM emp WHERE empId=id;
RETURN n;
END;
SELECT fun_return_emp_byId(1);
SET @n = fun_return_emp_byId(1);
SELECT @n;

```

知识点3 执行存储过程

知识点4 存储过程的删除

MySQL数据库存储过程的删除

语法

 drop procedure/function 存储过程名称

比如

dorp procedure proc_test

第7章 触发器

知识点1 前触发器和后触发器

MySQL数据库触发器

触发器是为响应某个事件而执行的任务。

事件包括添加、修改、删除操作，任务可以在操作前或操作后执行
触发器不能手动调用，创建好以后，会由mysql数据库自动触发并执行

分类

前触发器：before

在执行相应操作之前执行触发器中的代码 I

后触发器：after

在执行相应操作之后执行触发器中的代码

优点

- (1) 触发器自动执行，可以减少额外的操作
- (2) 触发器可以在更新数据库之前校验数据的合法性
- (3) 触发器可以强制引用完整性

知识点2 创建触发器

MySQL数据库触发器创建

语法

```
create trigger 触发器名  
before / after  
insert / update / delete  
on 表名  
for each row  
begin  
    逻辑代码  
end;
```

注意

在触发器中想获得更新前或更新后的数据，请使用:new.字段 或 old.字段
在触发器中不能使用select输出

MySQL数据库触发器删除

语法

```
drop trigger 触发器名
```

```
/*  
触发器  
*/  
#删除数据前，先把编号和姓名记录到log中  
CREATE TRIGGER tr_emp_delete  
BEFORE #操作之前执行  
DELETE /*AFTER INSERT/UPDATE/DELETE*/  
ON emp  
FOR EACH ROW  
BEGIN  
    DECLARE info VARCHAR(200);  
    DECLARE id INT;  
    SET id = old.empId;  
    SET n = old.empName;  
    SET info = CONCAT('ID:',id,',empName:'n);  
    INSERT INTO log1 VALUES(NULL,info);  
END;  
DELETE FROM emp WHERE empId=4;  
SELECT * FROM emp;  
SELECT * FROM log1;  
  
#修改
```

```

CREATE TRIGGER tr_emp_update
AFTER UPDATE
ON emp
FOR EACH ROW
BEGIN
DECLARE info VARCHAR(200);
DECLARE old_name VARCHAR(20);
DECLARE new_name VARCHAR(20);
SET old_name = old.empId;
SET new_name = new.empName;
SET info = CONCAT('旧名:',old_name,',新名:'new_name);
INSERT INTO log1 VALUES(NULL,info);
END;
UPDATE emp SET empName='yu12' WHERE empId=2;
SELECT * FROM emp;
SELECT * FROM log1;

```

知识点3 查看已有触发器

有两种方式

第一种: show triggers

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_cl
(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)

第二种:

```

select * from information_schema.triggers
information_schema是mysql中提供的数据库

```

TRIGGER_CATALOG	TRIGGER_SCHEMA	TRIGGER_NAME	EVENT_MANIPU	EVENT_OBJECT_	EVENT_OBJECT_
(Null)	(Null)	(Null)	(Null)	(Null)	(Null)

知识点4 删除触发器

```
drop trigger tr_emp_delete
```

第8章 数据库的事务与分布式一致性

事务: 之前做的银行转账事务案例, 需要InnoDB引擎支持事务

事务控制语句:

BEGIN 或 **START TRANSACTION** 显式地开启一个事务;

COMMIT 也可以使用 **COMMIT WORK**, 不过二者是等价的。**COMMIT** 会提交事务, 并使已对数据库进行的所有修改成为永久性的;

ROLLBACK 也可以使用 **ROLLBACK WORK**, 不过二者是等价的。回滚会结束用户的事务, 并撤销正在进行的所有未提交的修改;

SAVEPOINT identifier, **SAVEPOINT** 允许在事务中创建一个保存点, 一个事务中可以有多多个 **SAVEPOINT**;

RELEASE SAVEPOINT identifier 删除一个事务的保存点, 当没有指定的保存点时, 执行该语句会抛出一个异常;

ROLLBACK TO identifier 把事务回滚到标记点;

SET TRANSACTION 用来设置事务的隔离级别。InnoDB 存储引擎提供事务的隔离级别有 **READ UNCOMMITTED**、**READ COMMITTED**、**REPEATABLE READ** 和 **SERIALIZABLE**。

MYSQL 事务处理主要有两种方法:

1、用 **BEGIN**, **ROLLBACK**, **COMMIT**来实现

BEGIN 开始一个事务

ROLLBACK 事务回滚

COMMIT 事务确认

2、直接用 **SET** 来改变 MySQL 的自动提交模式：

SET AUTOCOMMIT=0 禁止自动提交

SET AUTOCOMMIT=1 开启自动提交

原子性：一个事务（**transaction**）中的所有操作，要么全部完成，要么全部不完成，不会结束在中间某个环节。

一致性：在事务开始之前和事务结束以后，数据库的完整性没有被破坏。

隔离性：数据库允许多个并发事务同时对其数据进行读写和修改的能力，隔离性可以防止多个事务并发执行时由于交叉执行而导致数据的不一致。事务隔离分为不同级别，包括读未提交（**Read uncommitted**）、读提交（**read committed**）、可重复读（**repeatable read**）和串行化（**Serializable**）

持久性：事务处理结束后，对数据的修改就是永久的，即便系统故障也不会丢失。

mysql> use RUNOOB;

Database changed

mysql> CREATE TABLE runoob_transaction_test(id int(5)) engine=innodb; # 创建数据表

Query OK, 0 rows affected (0.04 sec)

mysql> select * from runoob_transaction_test;

Empty set (0.01 sec)

mysql> begin; # 开始事务

Query OK, 0 rows affected (0.00 sec)

mysql> insert into runoob_transaction_test value(5);

Query OK, 1 rows affected (0.01 sec)

mysql> insert into runoob_transaction_test value(6);

Query OK, 1 rows affected (0.00 sec)

mysql> commit; # 提交事务

Query OK, 0 rows affected (0.01 sec)

mysql> select * from runoob_transaction_test;

+-----+

| id |

+-----+

| 5 |

| 6 |

+-----+

2 rows in set (0.01 sec)

mysql> begin; # 开始事务

Query OK, 0 rows affected (0.00 sec)

mysql> insert into runoob_transaction_test values(7);

Query OK, 1 rows affected (0.00 sec)

mysql> rollback; # 回滚

Query OK, 0 rows affected (0.00 sec)

mysql> select * from runoob_transaction_test; # 因为回滚所以数据没有插入

+-----+

| id |

+-----+

```
| 5 |
| 6 |
+-----+
2 rows in set (0.01 sec)
```

练习题

1、--- 给任职日期超过15年的员工加薪20%-- 注意:先备份表，然后对备份表进行操作

```
#给任职日期超过15年的员工加薪20%-- 注意:先备份表，然后对备份表进行操作
CREATE TABLE emp_c AS SELECT * FROM emp;#备份
#给员工工龄>15的加薪2%
UPDATE emp_c SET SAL=SAL*(1+20/100) WHERE (TO_DAYS(NOW())-
TO_DAYS(HIREDATE))/365>15;
UPDATE emp_c SET SAL=SAL*1.2 WHERE (TO_DAYS(NOW())-TO_DAYS(HIREDATE))/365>15;
SELECT * FROM emp_c;
SELECT * FROM emp;
```

给任职日期超过15年的员工加薪20% ——注意：先备份表，然后对备份表进行操作

```
USE sqldemo;
-- 先备份一下表
CREATE TABLE emp_bak AS SELECT * FROM emp;
UPDATE emp_bak SET sal = sal *1.2 WHERE (TO_DAYS(NOW())-TO_DAYS(hiredate))/365>15;
-- 查询修改数据
SELECT * FROM emp_bak;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	960.00	(NULL)	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1920.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1500.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	3570.00	(NULL)	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1500.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	3420.00	(NULL)	30

2、--- 求部门名称中带“A”字符的部门员工的工资合计、 部门人数

```
#求部门名称中带“A”字符的部门员工的工资合计、 部门人数
SELECT
    d.dname,
    SUM(e.sal) AS sumsal,
    COUNT(e.ename) AS countEmp
FROM emp e LEFT JOIN dept d
ON e.deptno = d.deptno
WHERE d.DNAME LIKE '%A%'
GROUP BY d.dname;
```


dname	sumsal	countEmp
ACCOUNTING	29025.00	14
OPERATIONS	29025.00	14
RESEARCH	29025.00	14
SALES	29025.00	14

3、--- 求出员工领导的薪水超过1700的员工名称和领导名称

#求出员工 领导的薪水超过1700

```
SELECT * FROM emp WHERE sal>1700;
```

#使用内连接查询

```
SELECT m.ename AS 员工名称,e.ename AS 领导名称
```

```
FROM emp m JOIN emp e
```

```
ON m.mgr = e.empno
```

```
WHERE m.sal>1700;
```

```
-----
```

```
SELECT
```

```
    nomal.ename,leader.ename AS boss ,leader.sal
```

```
FROM
```

```
    emp nomal
```

```
JOIN
```

```
    emp leader
```

```
ON
```

```
    nomal.mgr = leader.empno
```

```
WHERE
```

```
    leader.sal>1700;
```

员工名称	领导名称	ename	boss	sal
		SMITH	FORD	3000.00
		ALLEN	BLAKE	2850.00
		WARD	BLAKE	2850.00
JONES	KING	JONES	KING	5000.00
BLAKE	KING	MARTIN	BLAKE	2850.00
CLARK	KING	BLAKE	KING	5000.00
SCOTT	JONES	CLARK	KING	5000.00
FORD	JONES	SCOTT	JONES	2975.00
		TURNER	BLAKE	2850.00
		ADAMS	SCOTT	3000.00
		JAMES	BLAKE	2850.00
		FORD	JONES	2975.00
		MILLER	CLARK	2450.00

```

-- 求老板
SELECT emp.ename,emp.sal FROM emp WHERE emp.mgr IS NULL;
-- 求领导
SELECT DISTINCT boss.ename,boss.sal
FROM emp nomal
JOIN emp boss
ON nomal.`MGR`=boss.`EMPNO`
WHERE boss.sal >1700 AND boss.mgr IS NOT NULL;
-- 领导和老板加起来
SELECT DISTINCT boss.ename,boss.sal
FROM emp nomal
JOIN emp boss
ON nomal.`MGR`=boss.`EMPNO`
WHERE boss.sal >1700 AND boss.mgr IS NOT NULL
UNION
SELECT emp.ename,emp.sal FROM emp WHERE emp.mgr IS NULL;
-- 创建一个视图
CREATE VIEW leadername AS
SELECT DISTINCT boss.ename,boss.sal
FROM emp nomal
JOIN emp boss
ON nomal.`MGR`=boss.`EMPNO`
WHERE boss.sal >1700 AND boss.mgr IS NOT NULL
UNION
SELECT emp.ename,emp.sal FROM emp WHERE emp.mgr IS NULL;
-- 求员工
SELECT nomal.ename,nomal.sal FROM emp nomal WHERE nomal.ename NOT IN (SELECT
leadername.ename ename FROM leadername) AND nomal.sal>1700;
-- 创建员工视图
CREATE VIEW nomals
AS
SELECT nomal.ename,nomal.sal FROM emp nomal WHERE nomal.ename NOT IN (SELECT
leadername.ename ename FROM leadername) AND nomal.sal>1700;
-- 联合求解
SELECT * FROM leadername UNION |SELECT * FROM nomals;

```

4、--- 列出各种工作的最低工资及从事此工作的雇员姓名

列出各种工作的最低工资及从事此工作的雇员姓名

```
-- 求出各工作中
SELECT
    job,MIN(sal) AS minsal
FROM
    emp e
GROUP BY
    job
-- # 将上表当做临时表
SELECT
    e.sal,e.job,e.ename
FROM
    emp e
JOIN
    (
        SELECT
            job,MIN(sal) AS minsal
        FROM
            emp e
        GROUP BY
            job
    ) t
ON
    e.job=t.job AND e.sal = t.minsal;
```

#列出各种工作的最低工资及从事此工作的雇员姓名

#列出各种工作的工资

```
SELECT DISTINCT emp.`JOB`,emp.`SAL` FROM emp;
```

#最低工资

```
SELECT DISTINCT emp.`JOB`,MIN(emp.`SAL`) FROM emp GROUP BY emp.`JOB`;
```

#创建 最低工资 视图

```
CREATE VIEW min_sal AS
```

```
SELECT DISTINCT emp.`JOB`,MIN(emp.`SAL`) AS minsal FROM emp GROUP BY emp.`JOB`;
```

```
SELECT * FROM min_sal;
```

#事此工作的雇员姓名

```
CREATE VIEW min_sal_ename AS
```

```
SELECT e.sal,e.job,e.ename
```

```
FROM emp e
```

```
JOIN min_sal
```

```
ON e.`JOB`=min_sal.job AND e.`SAL`=min_sal.minsal;
```

```
SELECT * FROM min_sal_ename;
```

sal	job	ename
800.00	CLERK	SMITH
1250.00	SALESMAN	WARD
1250.00	SALESMAN	MARTIN
2450.00	MANAGER	CLARK
3000.00	ANALYST	SCOTT
5000.00	PRESIDENT	KING
3000.00	ANALYST	FORD

5、--- 列出薪金高于在部门20工作的所有员工的薪金的员工 姓名和薪金、 部门名称

```
#列出薪金高于在部门20工作的所有员工的薪金的员工 姓名和薪金、部门名称
SELECT emp.ename,MAX(emp.`SAL`) FROM emp WHERE emp.`DEPTNO`=20;
#
CREATE VIEW max_sal AS
SELECT MAX(emp.`SAL`) AS maxSal FROM emp WHERE emp.`DEPTNO`=20;
SELECT * FROM max_sal;
#
SELECT e.ename,e.sal,d.dname
FROM emp e
JOIN dept d
ON e.`DEPTNO`=d.`DEPTNO`
WHERE e.`SAL`>(SELECT MAX(emp.`SAL`) AS maxSal FROM emp WHERE emp.`DEPTNO`=20);
```

ename	sal	dname
KING	5000.00	ACCOUNTING

6、--- 列出至少有5个员工的所有部门

```
#列出至少有5个员工的所有部门
SELECT e.deptno,d.dname,COUNT(e.empno) AS count_empno
FROM emp e
JOIN dept d
ON e.`DEPTNO`=d.`DEPTNO`
GROUP BY e.`DEPTNO`
HAVING count_empno>5;
```

deptno	dname	count_empno
20	RESEARCH	5
30	SALES	6

7、--- 求平均薪水的等级最低的部门的部门名称

```
#求平均薪水的等级最低的部门的部门名称
#求各个部门平均薪水
CREATE VIEW avg_job AS
SELECT e.`DEPTNO`,AVG(e.sal) AS avgSal
FROM emp e GROUP BY e.`DEPTNO`;
#获取最低等级值4
CREATE VIEW avg_grade AS
SELECT t.`DEPTNO`,t.`avgSal`,s.`GRADE`
FROM avg_job AS t
JOIN salgrade s
ON t.avgSal BETWEEN s.losal AND s.hisal;
#组合查询
CREATE VIEW avg_grade AS
SELECT a.`DEPTNO`,MIN(a.`GRADE`)AS minGrade FROM avg_grade a;
-----
SELECT
    t.deptno,d.dname,t.avg_sal,s.grade
```

```

FROM (
    SELECT e.deptno,AVG(e.sal) AS avg_sal
    FROM emp e
    GROUP BY e.deptno
) t
JOIN salgrade s
ON t.avg_sal BETWEEN s.losal AND s.hisal
JOIN dept d
ON d.deptno=t.deptno
HAVING s.grade=(
    SELECT MIN(grade) AS min_grade
    FROM (
        SELECT t.deptno,t.avg_sal,s.grade
        FROM (
            SELECT e.deptno,AVG(e.sal) AS avg_sal
            FROM emp e
            GROUP BY e.deptno) t
        JOIN salgrade s
        ON t.avg_sal BETWEEN s.losal AND s.hisal
    ) t
);

```

☐	deptno	dname	avg_sal	grade
☐	30	SALES	1566.666667	3

8、--- 取得平均薪水最高的部门的部门编号

#取得平均薪水最高的部门的部门编号

#第一步：先求出每个部门的平均薪资，作为临时表 t

```

SELECT e.deptno,AVG(e.`SAL`) AS avg_sal
FROM emp e
GROUP BY e.`DEPTNO`;

```

#第二步：从第一步得出的每个部门平均薪资里求出最高值

```

SELECT
    MAX(a.avg_sal) AS maxsal
FROM (
    SELECT e.deptno,AVG(e.`SAL`) AS avg_sal
    FROM emp e
    GROUP BY e.`DEPTNO`
) a;

```

#第三步：用 临时表 t 再去匹配第二步得出的最大值，看哪个部门的平均薪资等于第二步的值

```

SELECT e.`DEPTNO`,AVG(e.`SAL`) AS avg_sal
FROM emp e
GROUP BY e.`DEPTNO`
HAVING avg_sal = (
    SELECT
        MAX(a.avg_sal) AS maxsal
    FROM (
        SELECT e.deptno,AVG(e.`SAL`) AS avg_sal
        FROM emp e
        GROUP BY e.`DEPTNO`
    )a);

```

DEPTNO	avg_sal
10	2916.666667

索引面试题

/*

1, 请问, 慢查询出现后如何调优?

慢查询出现后, 首先查看是否缺少索引

2, 请问, 应该使用哪种索引引擎?

innodb

3, 索引的常用形式是如何?

多字段联合索引

4, **id**自增设定是否合理?

合理, 因为自增索引为自然数, 显示为**1 2 3 4 5 6**, 无论是范围查询例如**id<6 and id>4**, 通过叶子节点的双向指针可以快地进行范围查询; 还是做精确查询, **21**亿个数只需查**3**次, 都是非常方便和迅速的, 要把方便做范围查询, 排序, 和兼有良好的精确查询效率都答出来

5, 应该推荐整型索引使用还是**uuid**唯一性字符索引使用?

整型更好, 方便做范围查询, **uuid**可以做唯一性的精确查询, 但是范围查询对于一串字符串来说并不友好, 而且整型更利于排序

6, **mysql**的索引是什么?

帮助**mysql**高效获取数据的数据结构

7, **mysql**的索引结构是什么? 为什么? 其他的参考结构差到哪里?

B+树 数据结构, **1**、检索速度快; **2**、排序快,

1. 单一节点存储更多的元素, 使得查询的**IO**次数更少。

2. 所有查询都要查找到叶子节点, 查询性能稳定。

3. 所有叶子节点形成有序链表, 便于范围查询。 , 其他结构如**B-**树在数据量过大时会变成链表

*/